16 位流水线 CPU 核心设计文档

默认 AC1　CPU v1.6　ISA v1.5　架构版 r5　2026 年 11 月 3 日

本规范描述面向 EES-331 教学平台的 16 位哈佛结构处理器。CPU 采用顺序单发射、硬连线控制和五级流水线，以明确的指令契约组织运算、存储和控制流，并使数据、操作意图与有效标签保持一致。

指令沿 IF、ID、EX、MEM、WB 推进。独立的同步 IM 和 DM 提供固定一拍读接口，级间载荷保存本条指令后续需要的信息。已就绪依赖通过前递解决，加载依赖通过互锁等待；跳转和故障按指令年龄取消年轻工作，停止标签随流水排空。

第一章定义指令与接口契约，第二章搭建数据通路和状态边界，第三章说明流水控制，第四章给出系统实现、功能证据和性能取舍。附录保存版本迁移、运行来源及复现入口。机器字、字地址和寄存器值默认用十六进制表示；程序周期及计数默认用十进制。

表 1 术语与阅读约定

术语	本文含义
AC1	当前默认实现的版本标识，CPU v1.6、ISA v1.5
IF / ID / EX / MEM / WB	取指、译码读源、执行、访存、写回五个阶段
D / X / M / W	分别与 ID、EX、MEM、WB 绑定的载荷或信号前缀
RF / IM / DM	寄存器组、指令存储器、数据存储器
CU / DP / CW	控制职责、数据通路、随指令传递的操作意图字段集合
valid / bubble / stall / flush	有效标签、无效空槽、保持等待、取消年轻指令

目录

1 设计目标与指令契约 4
1.1 架构规格与程序员可见状态 4
1.2 指令格式与立即数扩展 5
1.3 指令编码与架构语义 6
1.4 机器码编码示例 7
1.5 核心接口与同步存储契约 7
1.5.1 同步存储边沿契约 8
1.6 设计依赖与课程开发流程 9
2 数据通路与状态边界 10
2.1 五级数据通路总体拓扑 10
2.1.1 整体控制与数据通路架构 11
2.1.2 五级数据通路详细连接 12
2.2 核心状态部件与组合运算单元 13
2.2.1 PC 与分支目标计算 14
2.2.2 寄存器组与立即数扩展 14
2.2.3 多路选择器 14
2.2.4 ALU 与 FPGA 进位链加减核心 15
2.2.5 进位溢出与标志寄存器 16
2.3 四个流水边界与随行载荷 16
2.4 架构状态与统一五级执行 17
2.4.1 架构状态及其唯一更新责任 18
2.5 典型指令的数据路径 19
2.5.1 ADD 立即数 19
2.5.2 LDR 与 STR 20
2.5.3 CMP 与 JZ 21
3 流水控制与跨级协调 23
3.1 硬连线控制与随行操作意图 23
3.1.1 控制逻辑模型 23
3.1.2 从 ID 译码到随行控制字 23
3.1.3 ID 译码与指令操作意图 24
3.1.4 有效资格将操作意图转换为写入 25
3.2 数据冒险与前递互锁 26
3.2.1 已就绪操作数与标志的前递 27
3.2.2 数据相关与 load use 停顿 27
3.2.3 冒险分类与解决矩阵 28
3.3 控制事件仲裁与副作用保护 29
3.4 全局模式与排空停止 30
3.4.1 运行模式与事件定义 30
3.4.2 状态转移与当前输出 31
3.4.3 排空完成与复位返回 32
3.5 跨级协同的逐拍实例 32

3.5.1 相邻 ADD 的 M 到 EX 前递 32
3.5.2 加载到存储的一拍互锁 33
3.5.3 CMP 到 JZ 的 Z 前递与年轻取消 34
3.5.4 较老存储完成与 HALT 排空 34
3.5.5 较老 MEM 故障胜过年轻 EX 分支 35
4 系统集成与验证结果 36
4.1 RTL 模块组织与连接 36
4.1.1 逻辑职责与实际 RTL 映射 37
4.1.2 数据通路与控制连接 38
4.2 系统层次与启动观测 39
4.2.1 文件初始化与系统连接 39
4.2.2 稳定停止与公开结果接口 40
4.3 板级时钟复位与观测边界 41
4.3.1 时钟复位与观测连接 42
4.3.2 完整实现与时序覆盖 42
4.4 分层验证方法与证据范围 43
4.4.1 自检查测试方法 43
4.4.2 单元与流水覆盖总览 43
4.4.3 流水时序与指令执行检查 45
4.4.4 文件启动整机验证 50
4.4.5 EES331 实板功能验证 51
4.5 性能基线与统计口径 52
4.5.1 CPU 性能验证 52
4.5.2 固定程序与性能基线 53
4.6 隔离性能候选与取舍 54
4.6.1 前递 MUX 组合候选 54
4.6.2 WB 到 MEM 存储数据候选 54
附录 A 版本演进与证据索引 56
A.1 历史编码迁移与示例勘误 56
A.2 运行身份与历史证据 56
附录 B 复现与维护 58
B.1 复现入口与文档维护 58
B.2 验证范围与后续修改条件 58
附录 C 参考资料 59
C.1 课程与工程参考 59

1 设计目标与指令契约

先从程序员视角规定资源、编码和状态变化，再说明核心接口及同步存储的边沿契约。后续数据通路、控制与验证均以本章为依据。

1.1 架构规格与程序员可见状态

表 2 架构规格

项目	规定
数据与指令	数据 16 位 指令定长 16 位
通用寄存器	R0 至 R3 共 4 个 16 位寄存器；R0 普通可写
标志状态	Z 为等值比较结果；C 为算术进位或无借位；V 为有符号溢出
地址单位	一个地址对应一个 16 位字；顺序 PC 加 1
存储组织	Harvard 结构；IM 与 DM 独立；基线各 256 字
算术结果	保留低 16 位；按模 65536 计算
复位	高有效同步复位；PC、RF 及 Z/C/V 清 0；DM 内容保留

程序通过 R0 至 R3、PC、Z/C/V 和 DM 内容观察执行结果。四个通用寄存器均可读写；PC 按字寻址，顺序执行时加 1。Z 保存最近一次 CMP 的等值结果，C/V 保存最近一次 ADD 或 SUB 的算术状态。

IM 与 DM 各有 256 个 16 位字，合法字地址为 0000 至 00FF。地址运算先保留低 16 位，再检查是否落在这个范围；例如 0020 加−1 得到 001F，FFFF 加 1 回绕 0000，而 0100 和 FFFF 作为最终访问地址均越界。

同步复位使 PC、RF 和 Z/C/V 回到 0，并重新开始执行。DM 保留复位前已完成的写入；其启动初值由初始化镜像给出。故障采用停止保护，软件恢复执行需要复位。

1.2 指令格式与立即数扩展

表 3 指令字段

字段	位范围	位宽	用途
Opcode	[15:12]	4	选择 CPU 指令
Rd	[11:10]	2	目的寄存器或第一源；STR 的数据源
Rs	[9:8]	2	第二源或访存基址
imm8	[7:0]	8	无符号常数或有符号位移

```
word = (Opcode << 12) | (Rd << 10) | (Rs << 8) | imm8
```

程序镜像将未使用字段置 0，硬件执行时忽略这些字段。寄存器形式的 imm8、立即数形式中未使用的 Rs，以及分支和 HALT 未使用的寄存器字段，都采用这一约定。编码工具检查字段范围，并将负位移编码为 8 位补码。

编码表中，imm8 内的 x 表示由程序填写的数据位；未使用字段中的 x 或横线表示该字段不影响执行。镜像中仍写入确定的二进制值，未使用位按上述规则置 0。

表 4 各类指令的字段使用规则

指令形式	Opcode	Rd 字段	Rs 字段	imm8 字段
寄存器形式	0 2 3 4 6 A	目的或第一源	第二源	未使用 置 0
立即数形式	1 5 7	目的及需要时的旧值源	未使用 置 0	无符号常数
内存形式	8 9	LDR 目的或 STR 数据源	地址基址	有符号位移
分支形式	C D	未使用 置 0	未使用 置 0	C 绝对 u8 D 相对 s8
停止形式	E	未使用 置 0	未使用 置 0	未使用 置 0

保留编码 B/F 无数据操作数，执行时产生非法指令故障。立即数字段的解释取决于 Opcode：MOV、ADD、SUB 立即数和 JUMP 采用无符号 u8；LDR、STR 和 JZ 采用有符号 s8。

```
ZExt(imm8) = {8'h00, imm8}
SExt(imm8) = {{8{imm8[7]}}, imm8}
```

u8 取值为 0 至 255，s8 取值为−128 至 127。以 FF 为例，零扩展得到 00FF，符号扩展得到 FFFF。JZ 的相对位移以本条指令的 pc+1 为基准。

1.3 指令编码与架构语义

下面 d、s 分别是 Rd 和 Rs 索引；u8 范围为 0 至 255，s8 范围为−128 至 127；pc 指本条指令的 pc 标签。标志列中的“保持”表示不写对应架构状态。

表 5 指令编码与架构语义

Opcode	指令	操作数来源与结果去向	标志
0000 0	MOV Rd,Rs	R[d] ← R[s]	保持
0001 1	MOV Rd,#u8	R[d] ← ZExt(imm8)	保持
0010 2	AND Rd,Rs	R[d] ← R[d] AND R[s]	保持
0011 3	OR Rd,Rs	R[d] ← R[d] OR R[s]	保持
0100 4	ADD Rd,Rs	R[d] ← low16(R[d]+R[s])	更新 C/V
0101 5	ADD Rd,#u8	R[d] ← low16(R[d]+ZExt(imm8))	更新 C/V
0110 6	SUB Rd,Rs	R[d] ← low16(R[d]−R[s])	更新 C/V
0111 7	SUB Rd,#u8	R[d] ← low16(R[d]−ZExt(imm8))	更新 C/V
1000 8	LDR Rd,[Rs+s8]	EA=low16(R[s]+SExt(imm8))；R[d] ← DM[EA]	保持
1001 9	STR Rd,[Rs+s8]	同上生成 EA；DM[EA] ← R[d]	保持
1010 A	CMP Rs1,Rs2	Rs1 占 Rd 字段，Rs2 占 Rs 字段；Z ← (R[d]==R[s])；不写 RF	更新 Z
1011 B	Reserved	非法指令；不产生数据写入	保持
1100 C	JUMP #u8	下一执行目标=ZExt(imm8)	保持
1101 D	JZ #s8	Z=1 时目标=low16(pc+1+SExt(imm8))	保持
1110 E	HALT	先前指令完成后停止执行；后续指令不产生架构副作用	保持
1111 F	Reserved	非法指令；不产生数据写入	保持

ADD/SUB 只更新 C/V，因此 JZ 判断最近一次 CMP 留下的 Z。加法 C 表示无符号进位；减法 C=1 表示无借位。V 表示 16 位补码运算超出−32768 至 32767。C/V 目前通过调试接口观察，现行 ISA 没有读取它们的指令。

为了将架构副作用映射到译码控制，将 Opcode 分为七组：0 至 3 写 RF；4 至 7 写 RF 和 C/V；8/9 分别完成加载与存储；A 只更新 Z；C/D 改变控制流；E 停止执行；B/F 产生非法指令故障。各组均按表 5 规定保持其余状态。

1.4 机器码编码示例

表 6 机器字编码示例

汇编	Opcode	Rd	Rs	imm8	机器字
MOV R0,R1	0000	00	01	00000000	0100
MOV R0,#0x22	0001	00	00	00100010	1022
AND R0,R1	0010	00	01	00000000	2100
OR R0,R1	0011	00	01	00000000	3100
ADD R0,R1	0100	00	01	00000000	4100
ADD R0,#0x22	0101	00	00	00100010	5022
SUB R0,R1	0110	00	01	00000000	6100
SUB R0,#3	0111	00	00	00000011	7003
LDR R0,[R1+2]	1000	00	01	00000010	8102
STR R0,[R1+2]	1001	00	01	00000010	9102
CMP R0,R1	1010	00	01	00000000	A100
JUMP #0x20	1100	00	00	00100000	C020
JZ −2	1101	00	00	11111110	D0FE
HALT	1110	00	00	00000000	E000
STR R1,[R2+4]	1001	01	10	00000100	9604

机器字由 Opcode、Rd、Rs 和 imm8 分别移位后按位或得到。例如 MOV R0,R1 中 Rs=01，因此机器字为 0100；STR R1,[R2+4]为 9604。负位移−2 编码为 FE，JZ 的规范机器字为 D0FE。

1.5 核心接口与同步存储契约

接口方向以 CPU 核心为参考。clk 提供统一上升沿，rst 为高有效同步复位。IM 和 DM 均在该时钟域内工作，读延迟固定为一拍。正常取指和访存按下述边沿契约交换地址、数据和标签。

表 7 CPU 核心执行接口

信号	方向	位宽	功能与时序
clk	输入	1	唯一核心时钟 上升沿更新状态
rst	输入	1	高有效同步复位 优先级最高

信号	方向	位宽	功能与时序
im_addr	输出	16	取指字地址 来自 PC_fetch
im_en	输出	1	合格取指请求；stall/停止/取消时阻止错误取指
im_dout	输入	16	同步 IM 输出；配合 IF/ID 的 pc/valid 供 ID 使用
dm_addr	输出	16	MEM 的完整 16 位 EA；物理截位前检查范围
dm_en	输出	1	合格 MR 或 MW 访问使能
dm_we	输出	1	合格 STR 写使能；仅 MEM 有效访问时为 1
dm_din	输出	16	EX/MEM 保存的已转发 STR 数据
dm_dout	输入	16	同步 DM 读结果；供 WB 的 load 选择
halted	输出	1	HALT 完成排空后的停止状态 直到复位
fault	输出	1	故障完成排空后的停止状态 直到复位

表 7 列出 12 项执行接口，合计 87 位。cpu_system 将这些端口直接接到片上同步 IM/DM；观测端口见第 4.2 节。核心逻辑端口与板级引脚分属两个边界，板级连接见第 4.3 节。

1.5.1 同步存储边沿契约

表 8 同步存储读时序

周期位置	IM 路径	DM 读路径
周期 k 内	地址与 im_en 稳定	MEM 的 EA 与读使能稳定
k 结束上升沿	采样地址 同步登记 pc/valid	采样地址 同步登记 rd/写回控制
周期 k+1 内	dout 供 ID 译码 与 IF/ID 标签一致	dout 供 WB 选择 与 MEM/WB 标签一致
k+1 结束上升沿	ID/EX 捕获译码及源值	RF 捕获当前 WB 的 load 数据

同步 IM/DM 将读地址在上升沿采样，返回数据与同一条指令的标签进入下一阶段。返回端保持及流水边界的具体对应关系见第 2.3 节；访问资格和越界处理见第 3.3 节。

DM 写入在 MEM 结束上升沿发生。背靠背 STR 后同地址 LDR 必须读到已写的新值；本基线不会在同一个单端口周期同时请求读和写。IM 与 DM 独立，允许 IF 取指与 MEM 访存同周期发生。

1.6 设计依赖与课程开发流程

设计按依赖关系展开：先确定任务与接口，再定义 ISA 和架构副作用；随后分配运算、存储资源和流水阶段，规定级间载荷；在此基础上设计译码、前递、互锁和停止控制，完成 RTL 集成与分层验证，最后比较实现性能并进行实板验收。每一步都落实前一步的行为契约。

课程实施采用逐步集成：Pre2 完成 MUX 和加法器，Pre3 补齐 ISA 及关键部件，Pre4 连接数据通路，Pre5 完善流水控制，Pre6 整合系统、功能与性能证据。表 9 将课程成果映射到本文的主题章节。

表 9 课程任务与本规范的对应关系

课程阶段	应说明的成果	主要阅读位置
Pre2	mux、加法器及人工/EDA 验证	第二章部件契约；第四章验证方法及历史原始记录
Pre3	ISA、RF、ALU、PC、立即数扩展及验证	第一章指令契约；第二章资源和状态；第四章覆盖
Pre4	数据通路、接口、典型指令、源码与波形	第二章五级路径和边界；第三章逐拍例；第四章连接与证据
Pre5	CU 结构、工作原理、状态图	第三章控制层次、CW、仲裁、FSM 与并行实例
Pre6	设计流程、整机集成、功能和性能	本节设计依赖；第四章系统层次、功能和性能结果

2 数据通路与状态边界

本章沿数据的产生、保存和消费展开：先看五级拓扑与部件，再建立流水边界和状态更新规则，最后用典型指令追踪完整路径。

2.1 五级数据通路总体拓扑

五级流水线将取指、读源、执行、访存和写回分开，使不同指令在同一周期使用不同阶段的资源。IF 以 PC 向同步 IM 发出请求；ID 译码并读取 RF；EX 完成运算、地址或分支计算；MEM 连接 DM；WB 选择结果并更新 RF 和标志。

图 1 给出执行核心的整体连接，图 2 展开五级数据路径。阅读本章时先沿 IF 至 WB 追踪数值和标签；第三章再说明依赖、跳转和停止如何改变推进方式。

表 10 五级数据通路阅读提要

阶段或边界	主要数据流	本章阅读位置
IF	PC 字地址→同步 IM→指令	PC 与目标计算；四个流水边界
ID	指令字段→RF 两路读值与扩展立即数	寄存器组与立即数扩展
EX	操作数选择→ALU、比较器和分支目标	选择器、ALU 与标志候选
MEM	有效地址与 store_data→DM	统一五级执行契约
WB	运算结果或 load_data→RF 与 flags	架构状态和更新边沿
级间载荷	数据、控制意图与有效标签一起传递	四个流水边界的载荷清单

2.1.1 整体控制与数据通路架构

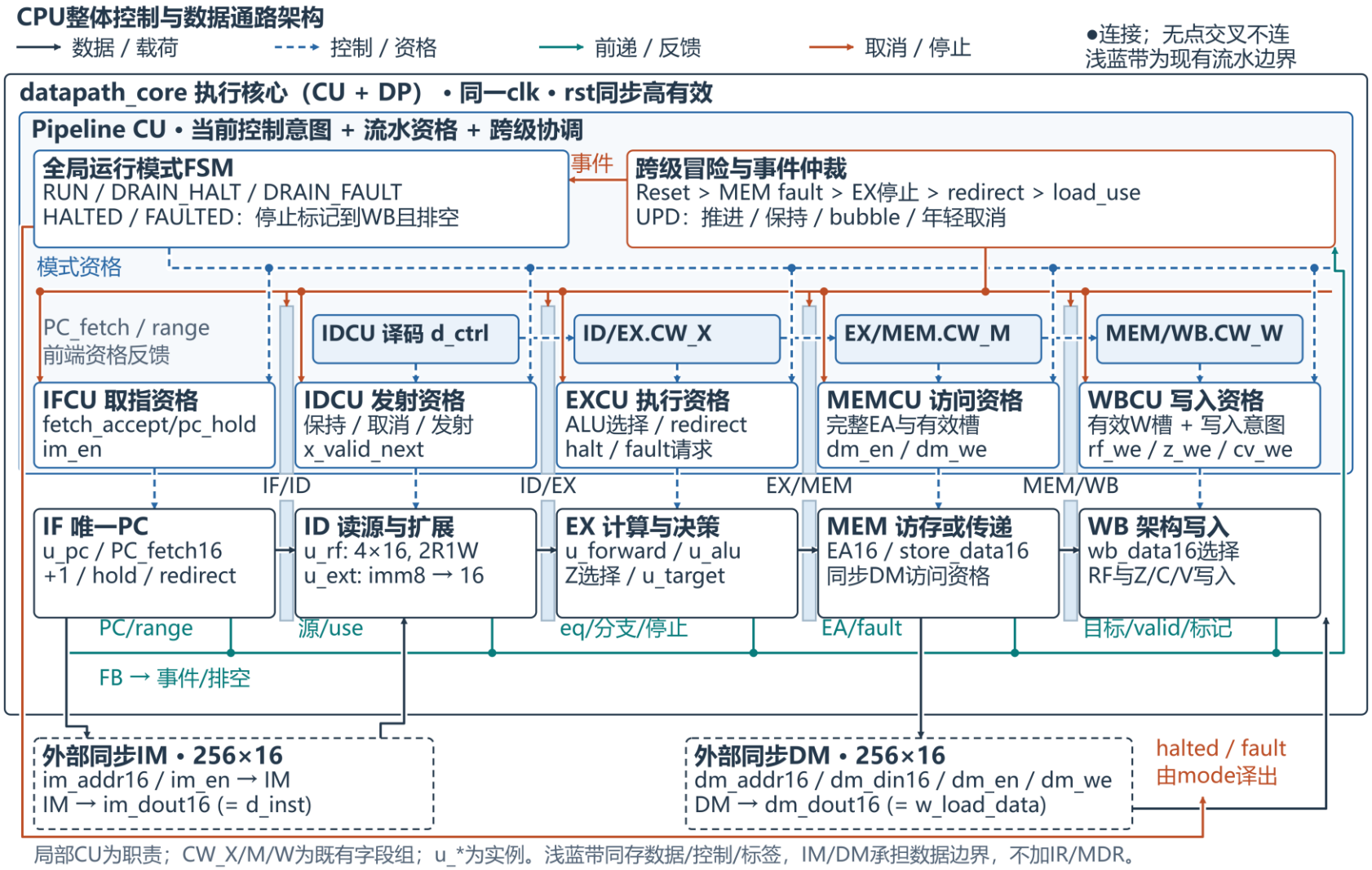


图 1 整体控制与数据通路架构。上层控制职责协调下层五级数据路径，浅蓝边界关联同一条指令的数据、标签与控制。

2.1.2 五级数据通路详细连接

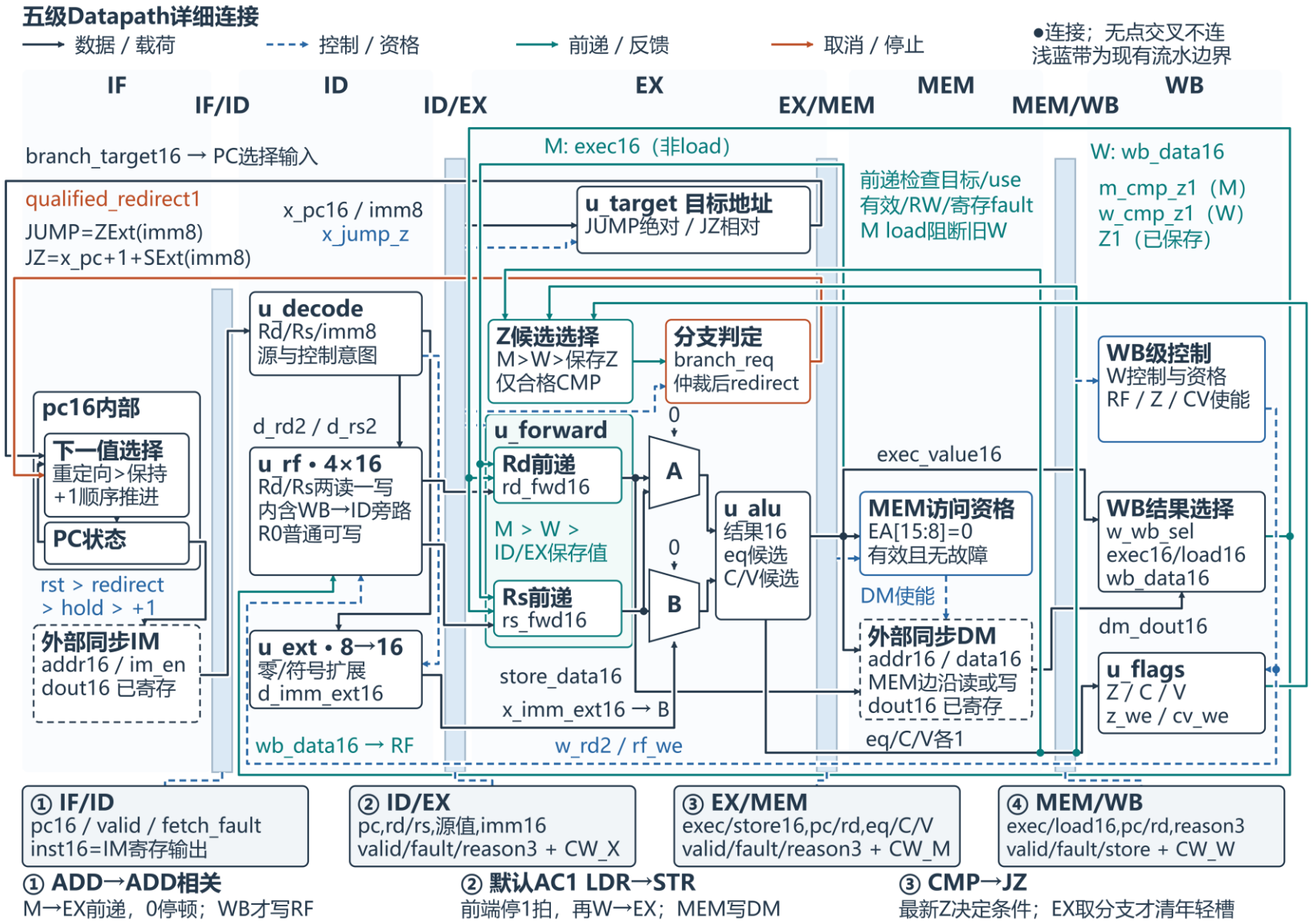


图 2 五级 Datapath 详细连接。目标计算与分支判定分开；M/W 前递、RF 写回及 Z 反馈均标出源端。浅蓝带为既有流水边界，完整字段见表 14；同步存储输出承担相应数据边界。

EX 先从前递网络取得 Rd 和 Rs 的当前值，再由 A/B 选择器组成 ALU 输入。算术结果和访存地址共用 exec_value；STR 同时将 Rd 的数据单独保存为 store_data，因此基址与写入数据可以独立选择。

WB 选择 exec_value 或同步 DM 返回值写入 RF。比较结果沿 CMP 的标签传递，供后续 JZ 选择最新 Z；分支目标则由本条指令的 pc 和立即数独立计算。前递优先级与推进控制见第 3.2 节。

2.2 核心状态部件与组合运算单元

表 11 主要模块接口与职责

模块	输入	输出	职责
isa_decode	inst[15:0]	rd/rs imm 控制字 illegal	将 4 位 Opcode 映射为内部选择及副作用意图
regfile4x16	clk rst we 两读地址及写地址各 2 位 wdata16	两路读值各 16 位	4×16 双读单写 含 WB 到 ID 旁路
imm_ext16	imm[7:0] sign_mode	value[15:0]	明确零扩展或符号扩展
operand_forward	实际源索引/use 保存值 M/W 生产者	rd_fwd16 rs_fwd16	为 EX 选择最新已就绪操作数
alu16	a16 b16 op3	result16 eq illegal carry overflow	组合运算 独立等值比较 产生 C/V 候选
pc16	clk rst hold redirect target16	pc16	复位 重定向 保持 顺序加 1
branch_target16	branch_pc16 imm8 relative	target16	JUMP 绝对目标或 JZ 相对目标
flags_reg	clk rst z_we cv_we z_in c_in v_in	Z C V	按掩码保存标志
hazard_control	各级 valid 实际源/use load 目标 停止及重定向事件	hold kill 取指资格	仲裁停顿 冲刷和排空
pipe_regs	clk rst 各级控制及载荷	四组流水载荷	保存数据 控制和有效性

表 11 按逻辑职责列出数据通路部件。各部件先规定输入、输出和状态更新规则，再由级间载荷将它们连接；实际 RTL 文件及内部逻辑的对应关系统一列在第 4.1 节。

2.2.1 PC 与分支目标计算

PC 保存唯一的前端取指地址。正常取指时加 1；接到合格重定向时装入目标；保持时继续提供原地址。每条指令另携带自己的 pc 标签，因此分支计算使用本条地址，而非已经超前的取指 PC。

JUMP 目标为 ZExt(imm8)；JZ 目标为 low16(pc+1+SExt(imm8))。branch_target16 组合计算目标，pc16 在上升沿按复位、重定向、保持、顺序加 1 的优先级更新。

2.2.2 寄存器组与立即数扩展

RF 有两个组合读端和一个同步写端，ID 通过 Rd、Rs 同时取得两路源值，WB 在上升沿更新选定寄存器。如果本拍 WB 与 ID 访问同一地址，读旁路直接返回当前写回数据，保证 ID 在该边沿捕获最新值；同步复位则将四个寄存器清 0。R0 与其他寄存器使用相同规则。

use_rd/use_rs 标明一条指令实际需要哪些源值：MOV 立即数只写 Rd，LDR 只读 Rs，STR 同时读 Rd 数据与 Rs 基址。这些掩码供第三章的相关检测使用。

立即数扩展器为组合逻辑，按本条指令的 sign_mode 形成 16 位值。ID 将扩展结果与操作意图一起送入 ID/EX，EX 使用保存值，从而保持立即数与指令标签一致。

2.2.3 多路选择器

表 12 数据通路选择编码

选择点	编码	输出用途
EX 转发选择	M 结果优先 W 结果次之 最后保存值	分别形成 rd_fwd 和 rs_fwd
alu_a_sel[1:0]	0=常数 0 1=rd_fwd 2=rs_fwd 3=保留	ALU a
alu_b_sel[1:0]	0=常数 0 1=rs_fwd 2=imm_ext 3=保留	ALU b
ALU result 选择	PASS_B ADD SUB AND OR	result16
wb_sel	0=exec_value 1=load_data	RF 写回 wdata
Z 有效值选择	M 级 ZW 候选 W 级 ZW 候选 架构 Z	JZ 条件
PC 下一值	合格 redirect 目标 否则 hold 或 pc+1	PC_fetch

MUX 是组合选择逻辑，不保存数据。先对 Rd/Rs 完成转发，再选择 ALU 输入。STR 将 rd_fwd 直接保存为 store_data，不把它误接为地址加法 a 输入。

人工设计的单比特 2 选 1 关系为 y=(NOT s AND d0) OR (s AND d1)，16 位选择器对每一位重复该关系。行为 HDL 可用条件运算符或完整 case 实现，结构 HDL 可用门电路或小 MUX 实例化；二者必须具有相同选择编码。保留选择码使用安全默认值，并由有效控制检查报告非法情况。

2.2.4 ALU 与 FPGA 进位链加减核心

ALU 使用独立的 3 位内部 op 选择功能：0 为 PASS_B，1 为 ADD，2 为 SUB，3 为 AND，4 为 OR。eq 始终比较 a 与 b；CMP 使用这条独立比较路径。保留 op 5 至 7 输出 result=0、illegal=1、carry=0 和 overflow=0。CPU 的 4 位 Opcode 通过译码映射到这些内部功能。

ADD 与 SUB 共用 17 位组合加减表达式。sub=0 计算 a+b，sub=1 计算 a+(~b)+1；第 16 位保留 carry，低 16 位保留 result。该运算只使用本条 a、b 和 sub，保存的 C 不参与输入。

```
b_mod = b ^ {16{sub}}
wide_sum = {1'b0,a} + {1'b0,b_mod} + {16'b0,sub}
{carry, result} = wide_sum;  // carry=wide_sum[16], result=wide_sum[15:0]
```

当前 AC1 将上述表达式交给综合器映射到 FPGA 专用进位链。加减接口沿用 addsub16_cla 名称；其具体例化关系和映射边界见第 4.1 节。下面保留课程两级 CLA 的原理与独立单元定义，用于对应早期人工设计和单元验证。

课程基础理论与独立验证单元采用四个 CLA4 组成 CLA16。令 p=a XOR b、g=a AND b、c0=cin；组内先并行形成传播 P 和生成 G，再由组间超前网络计算四组入口进位。以下 sum 公式使用 XOR 传播量。

```
sum[i] = p[i] ^ c[i]
c1 = g0 | (p0 & c0)
c2 = g1 | (p1 & g0) | (p1 & p0 & c0)
c3 = g2 | (p2 & g1) | (p2 & p1 & g0) | (p2 & p1 & p0 & c0)
P = p3 & p2 & p1 & p0
G = g3 | (p3 & g2) | (p3 & p2 & g1) | (p3 & p2 & p1 & g0)
c4 = G | (P & c0)
```

四组 P/G 记为 P0 至 P3、G0 至 G3。K0=cin，K1 至 K4 分别为 c4/c8/c12/c16；每组入口由顶层同时求得：

```
K1 = G0 | (P0 & K0)
K2 = G1 | (P1 & G0) | (P1 & P0 & K0)
K3 = G2 | (P2 & G1) | (P2 & P1 & G0) | (P2 & P1 & P0 & K0)
K4 = G3 | (P3 & G2) | (P3 & P2 & G1)
   | (P3 & P2 & P1 & G0) | (P3 & P2 & P1 & P0 & K0)
```

组 P/G 由 a、b 独立形成，K1 至 K4 通过组间超前逻辑求得，这些关系定义了保留 CLA 单元的结构。课件中的 OR 传播量也能与 g 形成等价进位函数；求和公式 sum=p XOR c 则需要使用上述 XOR 传播量。最终 AC1 的资源和速度由第四章的实际实现报告评价。

2.2.5 进位溢出与标志寄存器

AC1 的 carry 取 wide_sum[16]，等价于 16 位算术的 c16。ADD 时表示无符号精确和大于 65535；SUB 时定义为无借位，unsigned(a)≥unsigned(b)则为 1。若调试 borrow，SUB 下 borrow=NOT carry；overflow 与 Z 写入规则不变。

教材加减器还将 unsigned_overflow 定义为 carry XOR sub：ADD 时检测进位，SUB 时检测借位。它与本规格保存的原始 C 不是同一个信号；如展示该诊断量，应另命名 u_overflow 并注明解释。V 始终表示有符号溢出，不能将借位或最高位进位直接称为 V。

overflow 表示精确有符号运算超出−32768 至 32767，公式使用原始 b 的符号位：

```
ADD_V = ~(a[15] ^ b[15]) & (result[15] ^ a[15])
SUB_V =  (a[15] ^ b[15]) & (result[15] ^ a[15])
```

非算术操作的组合 C/V 输出为 0，但只在 CVW=1 时才更新保存的 C/V。CMP 更新 Z 并保持 C/V；LDR/STR 尽管用 ADD 算地址，也保持所有架构标志。flags_reg 的同步 rst 最高优先级，z_we 和 cv_we 分别写各自字段，同时有效时分别更新。

表 13 算术结果与进位溢出验收向量

运算	result	C	V	原因
FFFF+0001	0000	1	0	无符号越界 有符号−1+1 正常
7FFF+0001	8000	0	1	有符号正数越界
8000+8000	0000	1	1	两类越界同时发生
0005−0003	0002	1	0	无借位 无符号及有符号均正常
0000−0001	FFFF	0	0	借位 有符号结果−1 正常
8000−0001	7FFF	1	1	无借位 有符号负数越界
7FFF−FFFF	8000	0	1	借位 32767−(−1)越界
1234−1234	0000	1	0	无借位 等值减法

以上为验收预期值。result[15]表示结果位串的最高位，不能直接当 overflow；carry 与 overflow 分别检查。

2.3 四个流水边界与随行载荷

同步存储器在上升沿采样读地址，未接受新的读请求时保持返回数据。在本设计中，IM 的寄存输出直接提供 ID 指令，DM 的寄存输出直接提供 WB 加载数据；核心在同一边沿登记相应 pc、目的寄存器和控制标签，使返回数据与标签对齐。

因此，IF/ID 和 MEM/WB 的数据部分已由存储器返回端承担；在外侧再串接同沿 IR 或 MDR 会多出一拍。Load-use 保持期间，IM 必须令 en=0 以保存当前 ID 指令，同时保持它的 pc 和 valid。表 14 据此列出四个逻辑边界的完整载荷。

表 14 数据 CW 与元数据的级间载荷

边界	数据	元数据与标签	CW	保存位置
IF/ID	inst16 由同步 IM 的 dout 保存	d_pc16、d_valid1、d_fetch_fault1	尚未译码，无 CW_X	core 标签 18 位 + 外部 IM 输出 16 位；不另加 IR
ID/EX	x_rd_value16、x_rs_value16、x_imm_ext16	x_pc16、x_rd2、x_rs2、x_valid1、x_fault1、x_reason3	CW_X 18 位逻辑集合	core 现有寄存字段 91 位；imm_signed 已在 ID 消费，不继续保存
EX/MEM	m_exec_value16、m_store_data16、m_cmp_z1、m_c1、m_v1	m_pc16、m_rd2、m_valid1、m_fault1、m_reason3	CW_M 7 位逻辑集合	core 现有字段 65 位；exec 可作 ALU 结果或 EA，store 数据独立
MEM/WB	w_exec_value16、w_cmp_z1、w_c1、w_v1；load16 来自同步 DM 的 dout	w_pc16、w_rd2、w_valid1、w_fault1、w_reason3、w_store1	CW_W 5 位逻辑集合	core 字段 48 位 + 外部 DM 输出 16 位；不另加 MDR

字段在最后一次使用后即可结束传递：imm_signed 在 ID 完成扩展后消耗，源索引和 use 掩码保留到 EX，store_data 保留到 MEM。pc、valid、fault 和 3 位 reason 继续标识在途指令，reason 仅供诊断；w_store 记录 STR 的完成类型。有效位决定载荷是否可被消费，取消通过清 valid 实现。复位必须清 valid 和架构状态，未使用的数据位可以保留原值。

2.4 架构状态与统一五级执行

每条被接受且未取消的指令都按 IF→ID→EX→MEM→WB 推进。阶段内完成组合计算，结束上升沿登记下一阶段载荷或更新架构状态。一个阶段没有写入动作时，指令仍可有效传递；停顿会延长驻留时间，bubble 则以 valid=0 占据空槽。

表 15 各指令的有效阶段职责

编码	ID	EX	MEM	WB
0 MOV reg	读 Rs	PASS_B Rs	有效传递，无 DM	写 Rd，无 flags

编码	ID	EX	MEM	WB
1 MOV imm	无实际 RF 源；ZExt	PASS_B imm	有效传递，无 DM	写 Rd，无 flags
2 AND / 3 OR	读 Rd/Rs	两源前递，位逻辑	有效传递，无 DM	写 Rd，保持 flags
4 ADD / 6 SUB reg	读 Rd/Rs	加减 result/C/V	有效传递，无 DM	写 Rd/C/V，Z 保持
5 ADD / 7 SUB imm	读 Rd；ZExt	加减以 imm 为 b	有效传递，无 DM	同上
8 LDR	只读 Rs；SExt	基址前递，EA	合法 DM 同步读	load 写 Rd，flags 保持
9 STR	Rd 数据/Rs 基址；SExt	两源分别前递	合法 DM 写一次；valid 继续	store/pc/valid 保留，无写入
A CMP	读 Rd/Rs	eq 产生 cmp_z	cmp_z/ZW 有效传递	只写 Z
C JUMP / D JZ	无 RF 源；目标扩展	EX 合格 redirect；JZ 取最新 Z	分支自身有效传递，无 DM	有效完成，无写入
E HALT	停止意图	按年龄确认 DRAIN	停止 token 有效传递	消费 token 且下一拍空才停止
B/F 与 fetch fault	故障标签	未被取消则确认 fault	无副作用 token 传递	消费 token，FAULTED

RF 和 Z/C/V 在 WB 结束边沿更新；DM 的 STR 写入在 MEM 结束边沿发生。STR 到 WB 时仅消费完成标签，该拍 DM 仍可服务另一条 MEM 指令。分支在 EX 改变后续取指路径，自身继续到 WB。停止与故障的排空规则见第 3.4 节。

2.4.1 架构状态及其唯一更新责任

流水线中的运算结果和标志候选只有在规定写入边沿才改变架构状态。ID 读取与 WB 写入的是同一份 RF；EX 读取与 WB 更新的是同一份架构 Z/C/V。M/W 保存的 cmp_z、c 和 v 是尚待提交的候选值。

表 16 归纳各类状态的更新责任。STR 在 MEM 完成唯一一次 DM 写入，随后以有效标签进入 WB；其他有效指令也按表 15 的规定完成后续阶段。观测状态的连接另见第 4.2 节。

表 16 状态对象和唯一更新责任

对象	实际保存位置	更新与使用规则
PC	u_pc.pc，16 位	唯一取指地址状态；Reset/redirect/hold/+1 按资格采样
架构 RF	u_rf.regs，4×16 位	ID 两读、WB 一写；内部 WB→ID 旁路；R0 普通可写

对象	实际保存位置	更新与使用规则
架构 flags	u_flags.Z/C/V	WB 按 z_we/cv_we 独立写；EX 只读取或选择较新 CMP 候选
D/X/M/W 载荷	core 现有流水字段及同步 IM/DM 输出	数据、CW 和标签属于同一条指令；一个字段只由其所属时序块更新
诊断原因与停止现场	core 的原因/观测时序块	reason 随标签推进；stop_pc/reason 在停止边沿锁存并保持至 Reset
RF 影子、最后 store	cpu_system 观察时序块	按合格事件记录结果；用于系统观测，复位清观察状态并保留 DM
板级复位及计数	ees331_application	请求同步、释放去抖和公开事件计数；不新增 CPU 流水级

2.5 典型指令的数据路径

以下算术、访存和比较分支用例分别说明 RF 写回、DM 访问以及 Z 与控制流的关系。相同规则也适用于其他采用同一路径的指令。

2.5.1 ADD 立即数

例：pc=0000，机器字 5001，初始 R0=7FFF、Z=1。目标是 R0=8000、C=0、V=1、Z 保持 1。

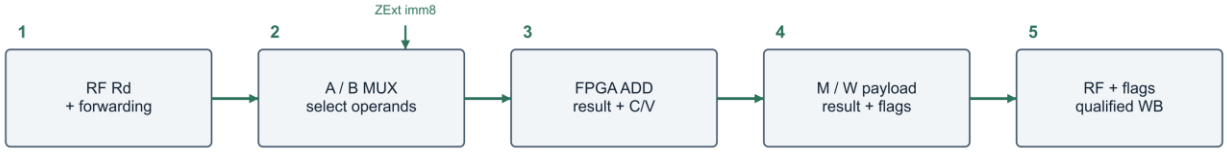


图 3 AC1 ADD 的数据流编号。①读取 Rd 并转发；②零扩展立即数经 B MUX；③17 位表达式映射 FPGA 进位链，生成低 16 位结果及 C/V；④候选值随流水传递；⑤WB 按原资格写 RF 和 C/V。

表 17 ADD 立即数逐拍操作

周期	阶段	数据与控制	结束边沿
C1	IF	im_addr=0000 合格 im_en=1	IM 返回 5001 登记 pc/valid
C2	ID	Opcode=5；use_rd=1 use_rs=0； imm_ext=0001；RW=1 CVW=1	保存 R0 旧值 源索引及控制
C3	EX	a=rd_fwd=7FFF b=0001 op=1；result=8000 C=0 V=1	EX/MEM 锁存 result 和 C/V 候选
C4	MEM	MR=MW=0；DM 不访问	载荷传入 MEM/WB
C5	WB	wb_sel=0；合格 RW/CVW 为 1；ZW=0	R0 写 8000 C/V 写 0/1 Z 保持

SUB 令 op=2，经同一 17 位补码加减表达式。MOV 选择 PASS_B；AND/OR 选择相应逻辑结果且 CVW=ZW=0，仍沿原普通写回路径。

2.5.2 LDR 与 STR

LDR 例：8BFF 表示 LDR R2,[R3−1]。设 R3=0020、DM[001F]=BEEF，地址流为 Rs → 转发 → A MUX，加 SExt(FF)=FFFF，得到 EA=001F；load 数据流为 DM → WB MUX → R2。

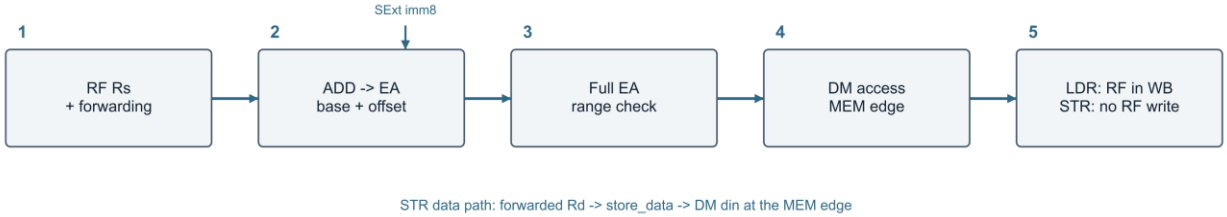


图 4 访存数据流。①转发 Rs 提供基址；②符号位移参与 ADD；③完整 EA 范围检查；④LDR 读取 DM 并选入 WB；⑤STR 另将转发 Rd 保存至 store_data 并在 MEM 写入。

表 18 LDR 与 STR 逐拍操作

周期	阶段	LDR 数据与控制	STR 数据与控制
C1	IF	取指并保存 pc	取指并保存 pc
C2	ID	use_rs=1；MR=RW=1 wb_sel=1；ZW=CVW=0	use_rd=use_rs=1；MW=1 RW=0；ZW=CVW=0
C3	EX	转发 Rs 加符号位移；EA 进 exec_value	同样生成 EA；rd_fwd 另锁入 store_data
C4	MEM	范围合法后 dm_en=1 dm_we=0；读结果随 rd/valid 供 WB	范围合法后 dm_en=dm_we=1；边沿写 DM
C5	WB	选 load_data，合格 RF 写入；EA 不写 RF	有效 STR 标签到 WB；不写 RF 或标志，不再次访问 DM

以 9900（STR R2,[R1+0]）为例，Rd 提供 R2 的写入数据，Rs 提供 R1 基址，两者分别前递。LDR 的返回值到 MEM 结束后才就绪，因此相邻消费者需要第三章规定的一拍互锁。

2.5.3 CMP 与 JZ

例：pc0 装 A100，pc1 装 D001，pc2 装 1C55，pc3 装 E000；初始 R0=R1=0001。CMP 产生 1，JZ 目标=0001+1+0001=0003，跳过 pc2 的 MOV R3,#0x55。

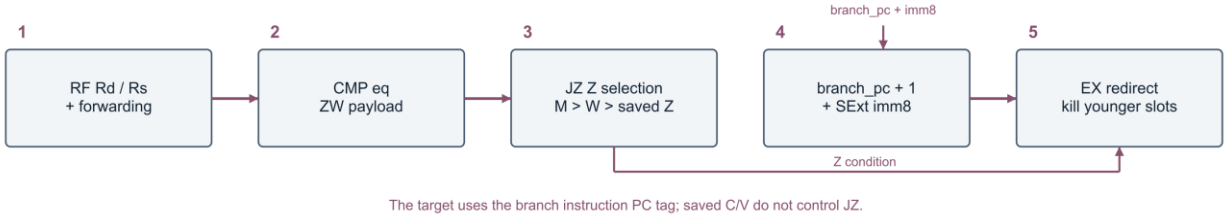


图 5 比较与分支数据流。①两个实际源转发至 eq；②CMP 结果随 ZW 进入 M/W；③JZ 选择最近较老 Z；④用分支自身 pc 计算目标；⑤EX 重定向 PC 并取消年轻槽。

表 19 CMP 与紧邻 JZ 逐拍操作

周期	CMP	JZ	前端与架构动作
C1	IF	未进入	取 pc0
C2	ID	IF	译码 CMP 并取 pc1
C3	EX 产生 cmp_z=1	ID	取 pc2 默认顺序推进
C4	MEM 携带 ZW 与 cmp_z	EX	JZ 由 M 级转发取得 Z=1；周期末 PC 改 0003，取消 ID 的 pc2 和当拍 IF
C5	WB 更新架构 Z=1	MEM 无数据写入	目标 pc3 进入 IF

相邻 CMP 与 JZ 通过候选 Z 连接，因此 JZ 可在 CMP 写回前判断条件。条件不成立时保持顺序取指；候选优先级和取消规则见第 3.2 节及第 3.3 节。

JUMP 不读标志，目标为 ZExt(imm8)，仍在 EX 执行同样的重定向与年轻槽取消。没有 delay slot。

3 流水控制与跨级协调

先说明单条指令如何携带操作意图，再处理多条在途指令的数据依赖，随后说明跳转、故障和停止如何按年龄协调，并以逐拍实例贯通这些机制。

3.1 硬连线控制与随行操作意图

3.1.1 控制逻辑模型

控制单元把一条指令的语义分解为可随流水推进的操作意图。ID 生成这些意图，后续各级结合本条有效资格和当前事件产生选择信号与写使能。图 6 用次态逻辑 F、控制状态 Q 和输出逻辑 G 说明这条控制链。

$$Q^{+} = F(Q, I) \quad O = G(Q, I)$$

周期内，F 根据当前 Q 与输入 I 计算下一状态，G 计算本拍动作；结束上升沿再由 Q 锁存。输入 I 包括指令、运算和地址检查结果、当前 Z 及较新的比较候选、复位和各级反馈。

Q 包含运行模式、各级有效和故障标签、随行控制及寄存器索引；数据载荷由相应流水边界保存。正常 RUN 中，五级并行处理不同指令，F 和 G 共同协调各级推进。完整 RTL 职责映射见第 4.1 节。

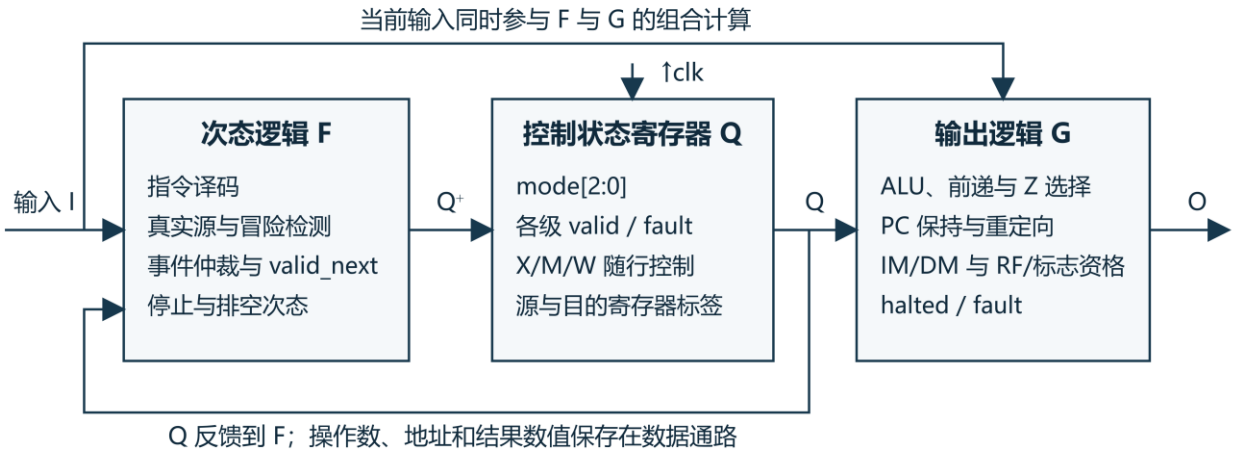


图 6 硬连线 CU 的 F/Q/G 逻辑模型。Q 反馈至 F，当前输入同时参与 F 和 G；图中方框表示现有控制职责。

3.1.2 从 ID 译码到随行控制字

ID 把 Opcode 和字段转换为操作数选择、真实源掩码及副作用意图。CW_X、CW_M、CW_W 是这些随行字段在 EX、MEM 和 WB 处的集合，总宽度分别为 18、7、5 位；字段随指令经过边界，并在最后使用后结束传递。

例如 imm_signed 在 ID 已用于扩展，无需带入 EX；reg_write 则要保存到 WB，等待写入资格。表 20 区分操作意图、数据及标签、本拍动作，随后两张译码表给出全部 Opcode 的具体取值。

表 20 三类信息的边界

类别	实际字段或信号	含义及约束
CW 操作意图	alu_op、a/b 选择、use_rd/rs、RW/MR/MW/ZW/CVW、wb_sel、jump/halt	描述这条指令后续需要什么；不等于当拍已经执行写入
数据与元数据	操作数/立即数/结果/store 数据；pc、rd/rs、valid、fault、reason、w_store	说明数据属于谁、是否有效和为何停止；候选 flags 是数据，valid/fault 是资格标签
当拍动作与下一状态	fetch_accept、pc_hold、qualified_redirect、dm_en/we、rf_we、z_we、cv_we、*_valid_next	由当前状态、CW 和相关事件计算；仅在规定边沿更新各自状态或存储器

3.1.3 ID 译码与指令操作意图

表 21 定义操作数路径。0/Rd/Rs/Imm 分别表示常数 0、已转发 Rd、已转发 Rs 和扩展立即数；imm_signed 对应 ID 扩展器 sign_mode。未使用选择项仍给安全的确定值。

表 21 指令译码操作数控制

Opcode	use_rd	use_rs	ALU a	ALU b	alu_op	imm_signed
0 MOV 寄存器	0	1	0	Rs	0	0
1 MOV 立即数	0	0	0	Imm	0	0
2 AND	1	1	Rd	Rs	3	0
3 OR	1	1	Rd	Rs	4	0
4 ADD 寄存器	1	1	Rd	Rs	1	0
5 ADD 立即数	1	0	Rd	Imm	1	0
6 SUB 寄存器	1	1	Rd	Rs	2	0
7 SUB 立即数	1	0	Rd	Imm	2	0
8 LDR	0	1	Rs	Imm	1	1
9 STR	1	1	Rs	Imm	1	1
A CMP	1	1	Rd	Rs	0	0
B 保留	0	0	0	0	0	0
C JUMP	0	0	0	0	0	0
D JZ	0	0	0	0	0	1
E HALT	0	0	0	0	0	0

Opcode	use_rd	use_rs	ALU a	ALU b	alu_op	imm_signed
F 保留	0	0	0	0	0	0

CMP 用合法 PASS_B 默认 op，但不使用 result；eq 比较选入的 Rd 与 Rs。JUMP/JZ 目标由 branch_target16 单独计算，输入为本条 pc 和 imm8，不能使用上表已置 0 的 ALU 结果。

表 22 定义写入及控制意图。RW/MR/MW/ZW/CVW 分别是 reg_write/mem_read/mem_write/z_write/cv_write；最终使能还需所属级 valid、故障和复位资格。

表 22 指令译码写入与控制掩码

Opcode	RW	MR	MW	ZW	CVW	其他控制
0 1	1	0	0	0	0	wb_sel=0
8	1	1	0	0	0	wb_sel=1
9	0	0	1	0	0	wb_sel=0
4 5 6 7	1	0	0	0	1	wb_sel=0
2 3	1	0	0	0	0	wb_sel=0
A	0	0	0	1	0	wb_sel=0
B F	0	0	0	0	0	illegal=1
C	0	0	0	0	0	jump_abs=1
D	0	0	0	0	0	jump_z=1
E	0	0	0	0	0	halt=1

各行未列出的 jump_abs/jump_z/halt/illegal 均为 0。组合译码先给默认值再 case 覆盖，不用 casex 掩盖未知 Opcode。CPU 保留 Opcode 与 ALU 非法 op 是两种错误来源，均须按有效指令检测。

Group 3 的 MemEn=1 只表示访存意图：LDR 的 MR/RW 为 1、MW 为 0；STR 的 MW 为 1、MR/RW 为 0。物理 dm_en/dm_we 须在 MEM 结合完整地址与有效资格产生。Group 7 的 1?11 只匹配 1011/1111，即 B/F；不能只检测低两位 11，否则会把合法 OR 和 SUB 立即数误判为非法指令。

3.1.4 有效资格将操作意图转换为写入

reg_write=1 表示这条指令需要更新 RF。当它到达 WB 且有效、无故障、未复位，并处于允许推进的模式时，WBCU 才产生 rf_we。相同规则将 Z/CV 写意图转换为独立使能。

MEM 按本条访存意图、有效标签和完整地址产生 dm_en/dm_we。具体资格表达式与同拍事件的优先级统一见第 3.3 节。

3.2 数据冒险与前递互锁

多条在途指令共享架构寄存器，但结果在不同阶段就绪。已就绪结果通过前递或 RF 旁路供给消费者；尚未就绪的加载结果通过前端互锁等待一拍。图 7 与表 23 概括这套协作关系。

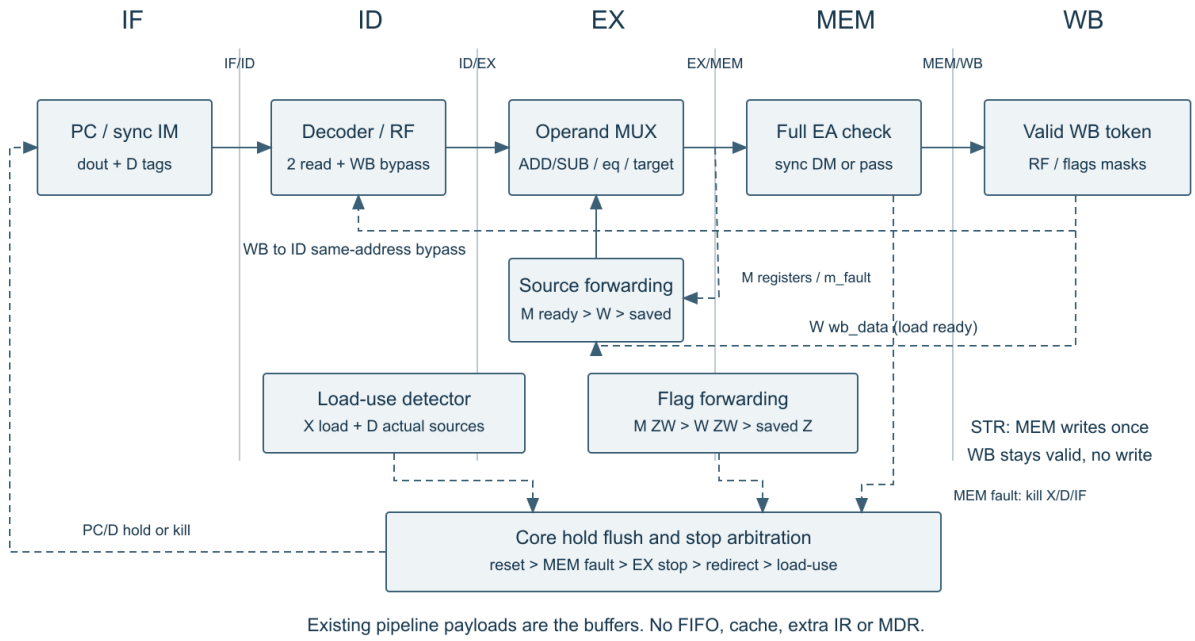


图 7 AC1 流水 Buffer 与 Interlock。M 寄存数据、valid/RW/就绪与 m_fault 用于源选择；完整 EA 检查用于 DM 保护及 MEM fault 取消，独立于前递数据 MUX。停顿、Flag forwarding 和年轻取消不变，不增加 FIFO/cache。

表 23 八项机制的实体与信号归属

机制	实现位置	主要接口与动作
HDU	core 内逻辑；未新增实体	X load/rd2 与 D 源/use/valid/fault；load_use1
操作数前递	operand_forward 原样复用	M/W 目标及资格；每路 sel2、fwd16、blocked1
WB 到 ID 旁路	RF 原样复用	rf_we、w_rd2、wb_data16，同址组合返回
Load-use interlock	core 内逻辑复用	PC/D hold，IM en=0，X bubble，M/W 推进
Flag forwarding	core 内 Z 选择复用	M/W ZW 候选及保存 Z；z_effective1
Redirect / Flush	core 仲裁复用	target16；取消 D/IF，MEM fault 另取消 X
Buffer / valid	现有流水载荷，core 最小修改	w_valid_next=m_valid；w_store1；无 FIFO/额外 IR/MDR
Stop / Reset	原 RUN/DRAIN/HALTED/FAULTED	停止 token+valid_next；按年龄仲裁，rst 最高

3.2.1 已就绪操作数与标志的前递

相邻指令常在生产者尚未写回时使用其结果。EX 因此先比较源索引，再从最近的有效生产者取数：M 级非 load 结果优先，其次 W 级 wb_data，最后使用 ID/EX 保存值。Rd 和 Rs 两路独立判断，R0 与其他寄存器相同。

当 WB 写入与 ID 读源同拍发生时，RF 内的显式旁路让 ID 直接取得 wb_data。生产者与消费者之间隔两条正常指令时，该路径把最新值捕获进 ID/EX。

JZ 使用独立的 Z 选择网络：M 级有效 CMP 候选优先于 W 级 CMP 候选，其次为架构 Z。较新的 0 也覆盖较旧的 1；中间的 ADD 或其他不写 Z 的指令不会挡住较老 CMP。C/V 没有软件读取者，因此当前 ISA 不产生 C/V 读后写依赖。

3.2.2 数据相关与 load use 停顿

LDR 在 EX 只形成地址，DM 数据在 MEM 结束边沿才返回。相邻消费者若立即进入 EX，就拿不到该值。因此，即使 M 目的寄存器匹配，也不能转发其地址，或越过这个较新 load 去取 W 中的旧同名值；相关检测需提前在 X 级 load 与 D 级真实源之间完成。

```
load_use = x_valid && !x_fault && x_mem_read && d_valid && !d_fault
       && ((d_use_rd && d_rd == x_rd) || (d_use_rs && d_rs == x_rd))
```

当 load_use 成立且没有更高优先级事件时，PC 和 IF/ID 保持，im_en=0，下一拍 EX 插入 bubble。load 与更老的 MEM/WB 继续推进，消费者延后一拍进入 EX，届时从 W 取得返回数据。

表 24 load use 停顿与转发

周期	LDR R0,[R1]	ADD R2,R0	关键动作
C1	IF	未进入	正常
C2	ID	IF	正常
C3	EX	ID 保持	检测 load use；阻止覆盖 IF/ID；下周期 EX 插 bubble
C4	MEM	ID	DM 在结束边沿返回数据 消费者进入 EX
C5	WB	EX	从 W 的 load_data 转发 a/b 对应实际源
C6	已完成	MEM	普通结果传递
C7	已完成	WB	写 R2 及本条 C/V

相邻 LDR→CMP、LDR→STR 基址和 LDR→STR 数据均按此规则停一拍。MOV 立即数只写目的而不读取旧值，目的同名不会造成停顿。实际停顿事件的资格与较老故障仲裁见第 3.3 节。

3.2.3 冒险分类与解决矩阵

表 25 按真实源角色归纳冲突。g 表示生产者与消费者之间的独立指令数；正常推进下，非 load 生产者在 g=0 时走 M 前递，g=1 走 W 前递，g=2 通过 WB 到 ID 旁路保存。遇到额外停顿时，选择以当拍实际所在级和就绪状态为准。完整用例索引见附录。

表 25 当前 ISA 冒险类别与真实用例

指令关系或事件	解决机制	额外停顿或处理
ALU/MOV→ALU、MOV 寄存器、CMP	真实 Rd/Rs 各自选择最近的已就绪结果	0 拍；相邻 M 前递，隔一条 W 前递
ALU/MOV→LDR/STR 基址	前递 Rs 后与符号位移求和	0 拍
ALU/MOV→STR 数据	前递 Rd 并单独保存 store_data	0 拍
LDR→MOV 寄存器、ALU、CMP	D 检测真实源，等待 load 返回后从 W 前递	相邻 1 拍，其余正常距离 0 拍
LDR→LDR/STR 基址	相同互锁应用于 Rs	相邻 1 拍
LDR→STR 数据	相同互锁应用于 Rd	相邻 1 拍
Rd=Rs、R0 或多个生产者同名	两路独立判断，最近生产者优先；未就绪 M load 阻断旧 W	按生产者就绪决定 0 或 1 拍
WB 与 ID 同址	RF 内部显式旁路	0 拍
只写不读的假相关	只比较 use_rd/use_rs 允许的源	0 拍
CMP→JZ 及连续 CMP	M 候选 Z>W 候选 Z>架构 Z；新 0 覆盖旧 1	0 数据停顿
JUMP 或 JZ 成立	EX 重定向并取消 ID/IF 年轻槽	2 个取指槽开销；不成立继续
MEM 故障与年轻 EX 事件	按年龄取消年轻动作并保留老 WB	事件仲裁，见第 3.3 节
HALT 或 EX 故障	老 M/W 继续，停止标记排空	停止完成条件见第 3.4 节
STR→同地址 LDR、连续访存	按 MEM 顺序访问，后读见前写的新值	0 结构停顿
IF/MEM 并发、RF 读写	IM/DM 分离；RF 两个读端、一个写端	资源容量满足单发射
WAW、WAR	固定顺序 WB；老指令 EX 取源早于年轻 WB	当前结构排除，无需额外 scoreboard

顺序单发射与固定阶段保证写回顺序。WAW 不会重排两次写入，WAR 不会让年轻写入抢在老指令取源之前；当前机制主要解决 RAW 和控制流变化。

同一 X 载荷不可能同时是分支和 LDR，因此合法 branch_req 与 load_use 不会同时成立。需要仲裁的是较老 MEM 故障与年轻 EX 分支、HALT 或 load-use；此时老故障优先，原始相关条件即使为 1，也不计为已执行的正常等待。

3.3 控制事件仲裁与副作用保护

跳转、停止、故障和停顿都可能要求改变流水线推进。仲裁首先比较指令年龄：较老指令的事件先确定年轻指令是否仍有执行资格；同步复位则覆盖全部在途状态。表 26 列出统一顺序。

表 26 流水事件的年龄与优先级

优先级	事件	前端与年轻指令	较老级
1	同步复位	PC/valid/停止状态清 0	RF 和 Z/C/V 清 0 复位边沿禁止 DM 写
2	MEM 自身故障	停取指 取消当前 EX 及更年轻动作	故障 MEM 不访问 老 WB 仍完成
3	EX 故障或 HALT	停取指 取消两个年轻槽	老 MEM/WB 继续 完成后停止
4	EX 合格重定向	PC 取目标 清年轻槽并取消错误路径 IF	老 MEM/WB 完成
5	load use	PC/IF/ID 保持 ID/EX 插 bubble	load 及老 MEM/WB 推进
6	正常推进	逐拍接收载荷	逐拍完成

分支在 EX 确认后改变 PC 并清除年轻 ID/IF 标签，较老 M/W 照常完成。保留 Opcode 或取指越界先作为标签进入流水线，只有仍有效并到达 EX 时才确认故障，因此它们可以被更老分支取消。

取指地址越界时关闭物理 im_en，同时登记该 pc 与 fetch_fault。DM 在截取 8 位物理地址前检查完整 16 位 EA，只有有效访存才执行范围检查；普通 ALU 结果超过 00FF 不产生访存故障。

MEM 确认故障后，禁止故障自身访问并取消年轻 EX 及前端；更老 WB 仍可完成。EX 故障或 HALT 只取消年轻槽，老 MEM 的合法 STR 仍可写入。故障和 HALT 如何保留停止标签并结束排空，见第 3.4 节。

物理写入使能由随行意图与本级资格相与。mode_active 覆盖 RUN 与两种 DRAIN，保证排空期间老指令继续；无效、故障或已取消标签不能产生 RF、DM、标志写入或新的控制事件。

```
mode_active = (mode == RUN || mode == DRAIN_HALT || mode == DRAIN_FAULT)
wb_ok = mode_active && w_valid && !w_fault
rf_we = !rst && wb_ok && w_reg_write
z_we  = !rst && wb_ok && w_z_write
cv_we = !rst && wb_ok && w_cv_write
mem_ok = !rst && mode_active && m_valid && !m_fault && m_address_in_range
dm_en = mem_ok && (m_mem_read || m_mem_write)
dm_we = mem_ok && m_mem_write
```

WB 的合法完成标签与实际 RF 写入分别判断：STR、分支、CMP 和 HALT 都可以合法完成，而未必写 RF。STR 已在 MEM 写入 DM，到 WB 仅计一次完成；故障 token 不计入合法完成数。

AC1 的 MEM 到 EX 前递控制只使用已寄存 m_fault，地址越界组合判定 m_access_fault 独立用于 DM 保护和年龄取消。这使 16 位地址检查不再串入 EX 前递 MUX 与 ALU 的组合路径。该划分仍保证正确取数：可在 M 前递的非 load RF 生产者不访问 DM；STR 不写 RF；LDR 本就未就绪并阻断旧 W。一旦新 MEM 故障成立，年轻 EX 结果被取消，老 WB 继续完成。

同步复位在上升沿清 PC、RF、Z/C/V、流水 valid 与停止状态，并禁止当拍 DM 写入。此前完成的 DM 写入保留；每级复位都先于保持和推进，外部按键的同步与释放处理由板级实现。

3.4 全局模式与排空停止

3.4.1 运行模式与事件定义

HALT 或故障确认后，CPU 需要停止接收年轻指令，并让老指令和停止标签完成余下阶段。全局模式以 RUN、DRAIN_HALT、DRAIN_FAULT、HALTED、FAULTED 管理这一过程；编码分别为 000、001、010、011、100。正常推进、互锁和分支都在 RUN 内进行。

输入事件采用已获资格的内部判定：E=m_fault_event||x_fault_event，H=qualified_halt，D=drain_done。E 包含有效 MEM/EX 故障；H 已经排除复位、自身故障及更老 MEM 故障；D 表示 WB 有效停止标记在本边沿被消费后，所有 valid_next 均为 0。rst 为同步高有效复位，优先于其他转移。

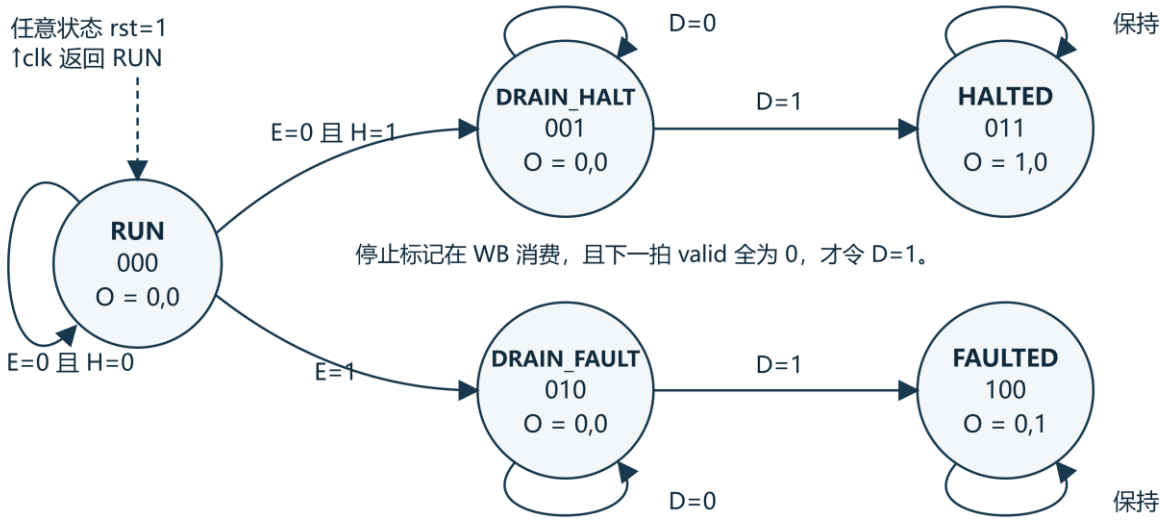


图 8 运行模式状态转移图。圆内 O=(halted,fault)为当前状态输出；箭头标转移条件。所有状态在 clk 上升沿更新，rst 优先。

3.4.2 状态转移与当前输出

halted=(mode==HALTED)，fault=(mode==FAULTED)，这两个输出满足 Moore 定义。表 27 输出列为边沿前的当前值；进入目标状态后再按新 mode 解释输出。RUN 不保证 im_en 恒为 1，取指仍受互锁、事件与地址资格控制。

表 27 CU 运行模式的完整转移与输出。除复位行外均取 rst=0。

当前 mode	输入条件	下一 mode	当前 O	边沿动作与结果
任意状态	rst=1	RUN	依当前态	清 PC/RF/标志/valid，禁本拍写入；已写 DM 保留
RUN 000	E=1	DRAIN_FAULT	0,0	禁止故障副作用，按年龄取消年轻槽
RUN 000	E=0,H=1	DRAIN_HALT	0,0	停取指；老合格指令与停止标记继续
RUN 000	E=0,H=0	RUN	0,0	正常推进、互锁或分支取消
DRAIN_HALT 001	D=0	DRAIN_HALT	0,0	老合格指令与 HALT 标记继续
DRAIN_HALT 001	D=1	HALTED	0,0	消费 WB 标记；边沿后 O=1,0
DRAIN_FAULT 010	D=0	DRAIN_FAULT	0,0	老合格指令与故障标记继续
DRAIN_FAULT 010	D=1	FAULTED	0,0	消费 WB 标记；边沿后 O=0,1
HALTED 011	任意 E/H/D	HALTED	1,0	保持停止，禁止访问与写入

当前 mode	输入条件	下一 mode	当前 O	边沿动作与结果
FAULTED 100	任意 E/H/D	FAULTED	0,1	保持停止，禁止访问与写入
101/110/111	任意 E/H/D	FAULTED	0,0	非法模式禁访问/写入；本沿转故障停止

3.4.3 排空完成与复位返回

E/H 事件启动排空时，halted/fault 仍为 0。停止标签依次经过后续阶段，在 WB 被消费且下一状态流水线为空的上升沿，CPU 进入 HALTED 或 FAULTED。停止模式保持到同步复位，接口通过复位重新启动。

例如，HALT 前的 LDR 仍应写回，HALT 后的 STR 则已被取消。故障场景同样保留较老的合格操作；第 3.5 节给出这两类情况的逐拍实例，定向测试证据集中列于第四章。

固定一拍存储使 EX/MEM 故障在当前周期内确认。较老 MEM 故障与年轻 EX HALT 同拍时直接选择 DRAIN_FAULT；HALT 一旦被接受，较老指令已通过相应检查，因此 DRAIN_HALT 内不会再出现新的较老故障。此时状态图无需 DRAIN_HALT 到 DRAIN_FAULT 的转移。可变延迟或延迟错误响应需要另行修订存储及仲裁契约。

排空判定使用下一状态有效位：stop_pending 表示当前处于任一 DRAIN，w_stop_token 表示 WB 保留 HALT 或故障标签。停止请求已关闭前端取指，后端继续按正常边沿消费在途载荷。

```
w_stop_token = w_valid && (w_halt || w_fault);
pipeline_empty_next = !(d_valid_next || x_valid_next ||
                        m_valid_next || w_valid_next);
drain_done = !rst && stop_pending && w_stop_token && pipeline_empty_next;
```

停止标签在本边沿消费，w_valid_next 清 0；其余 valid_next 也为 0 时，同一边沿更新最终停止模式。这样既让停止标签经过 WB，又无需多等一拍。没有停止标签时，单纯“没有写入”不能触发停止。

非法二进制 mode 由默认分支转入 FAULTED，mode_active 关闭其访问及写入。原因编码和受控故障注入证据见表 36 及第四章；所有状态寄存器均在上升沿更新。

3.5 跨级协同的逐拍实例

以下五例依次展示前递、加载等待、比较分支、HALT 排空和故障年龄仲裁。阶段列表示周期结束边沿前的有效指令，IF 列为实际取指请求；最后一列给出该边沿动作。初值统一由测试台在首条取指前建立，记录来源见附录。

3.5.1 相邻 ADD 的 M 到 EX 前递

初始 RF：R0=0001, R1=0002, R2=0004, R3=0000；初始 Z/C/V=000。

```
00: ADD R0,R1 (4100)
01: ADD R2,R0 (4800)
02: HALT (E000)
```

表 28 相邻 ADD 的 M 到 EX 前递逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:ADD	—	—	—	正常推进
C3	02	01:ADD	00:ADD	—	—	正常推进
C4	03	02:HALT	01:ADD	00:ADD	—	正常推进
C5	—	03:RES	02:HALT	01:ADD	00:ADD	RF R0←0003；WB 写 C/V
C6	—	—	—	02:HALT	01:ADD	RF R2←0007；WB 写 C/V
C7	—	—	—	—	02:HALT	停止 HALTED

C4 第一条 ADD 在 M、第二条 ADD 在 X，最近 M 非 load 结果可直接前递；不等待 RF 在 C5 写入，也不增加停顿。最终 R0=0003、R2=0007。

3.5.2 加载到存储的一拍互锁

初始 RF：R0=0000, R1=0021, R2=0000, R3=0020；初始 Z/C/V=000。

DM[0020]=0007；STR 的独立基址为 R1=0021。

```
00: LDR R0,[R3+0] (8300)
01: STR R0,[R1+0] (9100)
02: HALT (E000)
```

表 29 加载到存储的一拍互锁逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:LDR	—	—	—	正常推进
C3	—	01:STR	00:LDR	—	—	保持 IF/ID；X 插 bubble
C4	02	01:STR	—	00:LDR	—	正常推进
C5	03	02:HALT	01:STR	—	00:LDR	RF R0←0007
C6	—	03:RES	02:HALT	01:STR	—	DM[0021]←0007
C7	—	—	—	02:HALT	01:STR	正常推进
C8	—	—	—	—	02:HALT	停止 HALTED

C3 保持 IF/ID 并给下一拍 EX 插入 bubble，LDR 继续进入 MEM；C5 的 STR 从 W 取得 load 数据，C6 写 DM，C7 消费 STR 的有效 WB 标签，随后 HALT 于 C8 完成。因此相邻加载依赖只增加一拍等待。

3.5.3 CMP 到 JZ 的 Z 前递与年轻取消

初始 RF：R0=0001, R1=0001, R2=0000, R3=0020；初始 Z/C/V=000。

```
00: CMP R0,R1 (A100)
01: JZ #+1 (D001)
02: STR R0,[R3+0] (9300)
03: HALT (E000)
```

表 30 CMP 到 JZ 的 Z 前递与年轻取消逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:CMP	—	—	—	正常推进
C3	02	01:JZ	00:CMP	—	—	正常推进
C4	—	02:STR	01:JZ	00:CMP	—	EX 重定向；取消年轻槽
C5	03	—	—	01:JZ	00:CMP	WB 写 Z
C6	04	03:HALT	—	—	01:JZ	正常推进
C7	—	04:RES	03:HALT	—	—	正常推进
C8	—	—	—	03:HALT	—	正常推进
C9	—	—	—	—	03:HALT	停止 HALTED

JZ 在 X 使用 M 中最近 CMP 的候选 Z=1，无需等待架构 Z 写回。pc2 的 STR 被取消，不得写 DM；CMP 仍在 WB 更新 Z。目标 pc3 的 HALT 按完整路径停止。

3.5.4 较老存储完成与 HALT 排空

初始 RF：R0=BEEF, R1=0021, R2=1234, R3=0000；初始 Z/C/V=000。

```
00: STR R0,[R1+0] (9100)
01: HALT (E000)
02: STR R2,[R1+0] (9900)
```

表 31 较老存储完成与 HALT 排空逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:STR	—	—	—	正常推进
C3	02	01:HALT	00:STR	—	—	正常推进
C4	—	02:STR	01:HALT	00:STR	—	DM[0021]←BEEF
C5	—	—	—	01:HALT	00:STR	正常推进
C6	—	—	—	—	01:HALT	停止 HALTED

C4 老 STR 写 DM 且年轻 HALT 在 X 接受停止，老指令不能被 HALT 取消；STR 在 C5 继续有效 WB 但不再写 DM，HALT 在 C6 的 WB 结束边沿使核心停止。

3.5.5 较老 MEM 故障胜过年轻 EX 分支

初始 RF：R0=0000, R1=0000, R2=0000, R3=0000；初始 Z/C/V=000。

```
00: MOV R1,#255 (14FF)
01: ADD R0,#1 (5001)
02: LDR R2,[R1+1] (8901)
03: JUMP #5 (C005)
04: STR R0,[R0+0] (9000)
05: HALT (E000)
```

表 32 较老 MEM 故障胜过年轻 EX 分支逐拍过程

周期	IF	ID	EX	MEM	WB	结束边沿与动作
C1	00	—	—	—	—	正常推进
C2	01	00:MOV	—	—	—	正常推进
C3	02	01:ADD	00:MOV	—	—	正常推进
C4	03	02:LDR	01:ADD	00:MOV	—	正常推进
C5	04	03:JUMP	02:LDR	01:ADD	00:MOV	RF R1←00FF
C6	—	04:STR	03:JUMP	02:LDR	01:ADD	RF R0←0001；WB 写 C/V；M 越界压制 EX 跳转；老 WB 保留
C7	—	—	—	—	02:LDR	停止 FAULTED

C6 的 M 级 LDR 完整 EA=0100，访问被禁止，年轻 X 级 JUMP 不获重定向；更老 W 级 ADD 仍写 R0 与 C/V。故障标签继续 W，在 C7 消费后 FAULTED。故障不会回滚已经完成的更老写入。

4 系统集成与验证结果

从 RTL 连接进入片上系统和板级实现，再按单元、流水、系统与实板组织功能证据，最后说明性能基线与两个候选的工程取舍。

4.1 RTL 模块组织与连接

执行核心由 datapath_core 集成数据路径与硬连线控制，系统层由 cpu_system 连接同步 IM/DM，板级由 ees331_top 连接时钟、复位、应用逻辑和 VIO。表 33 将本规范的逻辑职责映射到源码，随后片段展示关键连接。

Vivado/rtl 保留 13 份核心 RTL，系统/rtl 另有一份 cpu_system.sv。表中的 hazard_control、pipe_regs 和 IFCU 至 WBCU 属于 core 内部职责；操作数前递是 operand_forward 实体，WB 到 ID 旁路位于 regfile4x16。CW_X/M/W 对应现有标量字段集合。

表 33 当前源码与连接约定

源文件或模块	与本规格关系	接入要求
rtl/regfile4x16.v	已有 4×16 RF 含 WB 到 ID 旁路	we 接合格 rf_we；两读口接 Rd/Rs
rtl/pc16.v	已有 reset > redirect > hold > increment	redirect/hold 由资格与优先级控制生成
rtl/imm_ext16.v	已有零/符号扩展	sign_mode 来自本条新译码
rtl/branch_target16.v	已有绝对/相对目标计算	branch_pc 来自 EX 随行标签
rtl/alu16.v、addsub16_cla.v	AC1 用 17 位求和推断进位链；C/V 接口不变	core→alu16→addsub；cla16/cla4 仅独立单元
rtl/isa_decode.v	已采用 AND/OR=2/3、LDR/STR=8/9	控制表与独立合约均已遍历 65,536 字
rtl/flags_reg.v	已实现独立 Z/CV 写入和同步复位	继续核对保存字段及指令写掩码
rtl/datapath_core.sv	原流水、仲裁、停止不变；M 前递接寄存 m_fault	MEM 故障年龄取消仍保留；AC1 有限回归已绑定
rtl/sync_memories.v	已实现固定单周期同步 IM/DM	保持 en 保持及标签对齐；DM 不随复位清空
九个 TB 入口及两份.svh	合约、单元、主回归、压力、手写事件检查	独立 ISA 预期与手算边界、真值表前递及逐拍检查；绑定当前源码和日志

RF 与 PC 片段说明具体的同步写入、组合读旁路和 PC 优先级。外围使能按第三章资格形成，状态由唯一时序块驱动。

```
// regfile4x16.v  synchronous write and explicit WB-to-ID bypass
always @(posedge clk) begin
  if (rst) for (i=0; i<4; i=i+1) regs[i] <= 16'h0000;
  else if (we) regs[waddr] <= wdata;
end
always @* begin
  rdata_a = regs[raddr_a]; rdata_b = regs[raddr_b];
  if (we && !rst && waddr == raddr_a) rdata_a = wdata;
  if (we && !rst && waddr == raddr_b) rdata_b = wdata;
end
// pc16.v  redirect is already qualified by the control unit
always @(posedge clk) begin
  if (rst) pc <= 16'h0000;
  else if (redirect) pc <= target;
  else if (!hold) pc <= pc + 16'd1;
end
```

4.1.1 逻辑职责与实际 RTL 映射

表 34 控制所有权与 RTL 对应

职责/对象	信息或意图来源	仲裁与资格	实际驱动与边沿
全局运行模式	core 组合逻辑由当前 mode 及合格故障/HALT/drain_done 求 mode_next	Reset > MEM/EX fault > HALT；停止状态保持至 Reset	core 主时序块唯一更新 mode；非 IF 到 WB 的串行 FSM
指令译码	u_decode 产生源使用、ALU 选择和 RW/MR/MW/ZW/CVW 等意图；u_ext 在 ID 解释立即数	core 用 d_valid/d_fault 及发射资格屏蔽无效控制	译码为组合逻辑；意图在进入 ID/EX 的边沿锁存
IF 请求与 PC	EX 目标、mode、load_use 及事件共同提供取指资格	core 产生 qualified_redirect、fetch_accept、pc_hold 及 im_en	u_pc 唯一保存 PC；同步 IM 保存指令 dout；core 保存 D 标签
ID 保持与发射	ID 实际源与 EX load 目的比较形成 load_use	core 统一计算 d_valid_next/x_valid_next，非独立 ID 暂停 FSM	主时序块更新 D/X 有效位；停顿时 PC/IM/D 标签共同保持、X 注入 bubble

职责/对象	信息或意图来源	仲裁与资格	实际驱动与边沿
EX 操作数前递	u_forward 比较 use/源号及 M/W valid/fault/RW/readiness	最近 M 匹配优先；M load 未就绪阻断旧 W	u_forward 内比较/sel/blocked 属控制、16 位选择属 DP；core 再选 ALU 输入
Z 与分支资格	core 选择 M CMP 候选、W 候选、已保存 Z；u_target 计算目标	EX 产生 branch_req，再经年龄与故障资格得到 qualified_redirect	pc16 采样重定向；目标计算不决定是否跳转，CMP 候选不直接写 Z
MEM 访问与故障	M 控制、完整 EA、valid/fault、rst 和 mode_active	m_address_in_range/m_access_fault；老 MEM 故障取消年轻 EX/ID/IF	core 驱动 dm_en/dm_we；同步 DM 在 MEM 结束边沿读或写，STR 只写一次
WB 架构写入	W 控制、valid/fault、rst 和 mode_active；同步 DM 给出 load 数据	core 形成 wb_ok、rf_we、z_we、cv_we	u_rf 与 u_flags 各自唯一更新架构状态；W 级 store/branch 标签不产生 RF 写入
流水数据与 CW	各级数据和意图形成下一载荷	core 唯一的占用仲裁计算 valid_next 及 fault_next	core 主时序块更新 D/X/M/W 载荷；有效性掩码保证同一条指令的数据与控制对齐
原因与停止现场	ID/EX/MEM 识别 reason；WB 停止 token 与 pipeline_empty_next 决定 drain_done	诊断使用同一有效标签；取消的年轻指令不保留现场	core 另一个时序块更新 x/m/w_reason 与 stop/last_completed，非全部流水字段同一 always
系统观察状态	cpu_system 采样合格 RF/MEM 写入事件	不参与任何执行选择或仲裁	system 时序块保存 RF 影子和最后 store；与架构 RF 不是两份执行资源

d_/x_/m_/w_分别绑定 ID/EX/MEM/WB。图中的 CW_X/M/W 汇总现有随行字段；IFCU 至 WBCU 表示各级控制职责。完整实现位置集中见第 4.1 节。图 1 中的 FB 为反馈组，UPD 为保持、取消及 valid_next 更新组。

4.1.2 数据通路与控制连接

下面片段把前递值接入 ALU、将 STR 数据独立保存、选择 WB 数据，并连接唯一 RF 和标志寄存器。当前 AC1 的算术层级为 datapath_core/u_alu→alu16/u_addsub→addsub16_cla 的 17 位表达式；保留 cla16/cla4_pg 供独立单元验证。

```
always @* begin
  alu_a = 16'h0000; alu_b = 16'h0000;
  case (x_alu_a_sel)
    2'd1: alu_a = rd_fwd;
    2'd2: alu_a = rs_fwd;
  endcase
  case (x_alu_b_sel)
    2'd1: alu_b = rs_fwd;
    2'd2: alu_b = x_imm_ext;
  endcase
end
alu16 u_alu(.a(alu_a), .b(alu_b), .op(x_alu_op),
  .result(alu_result), .eq(alu_eq), .illegal(alu_illegal),
  .carry(alu_c), .overflow(alu_v));
assign x_store_data = rd_fwd;
branch_target16 u_target(.branch_pc(x_pc), .imm(x_imm_ext[7:0]),
  .relative(x_jump_z), .target(branch_target));
assign wb_data = w_wb_sel ? dm_dout : w_exec_value;
regfile4x16 u_rf(.clk(clk), .rst(rst), .we(rf_we),
  .waddr(w_rd), .wdata(wb_data), .raddr_a(d_rd), .raddr_b(d_rs),
  .rdata_a(d_rd_value), .rdata_b(d_rs_value));
flags_reg u_flags(.clk(clk), .rst(rst), .z_we(z_we), .cv_we(cv_we),
  .z_in(w_cmp_z), .c_in(w_c), .v_in(w_v), .Z(Z), .C(C), .V(V));
assign dm_addr = m_exec_value;
assign dm_din = m_store_data;
```

wb_sel 随 load 标签进入 W；dm_dout 须遵守固定同步读契约并与 w_rd/w_valid 对齐。ALU 的 eq/C/V 和 ZW/CVW 由 EX/MEM、MEM/WB 保存后再接 flags_reg，不能直接把当前 EX 输出写入架构标志。操作数选择码 3 须由有效 EX 非法控制检测捕获；默认 0 赋值避免推断锁存器，不能取代错误检测。

流水 always 块使用非阻塞赋值；组合块覆盖全部情况并使用阻塞赋值。每个状态只由一个时序块驱动，load use 只保持前端，不用一个全局 enable 冻结全部级。

4.2 系统层次与启动观测

系统封装提供两项服务：按核心的一拍契约返回 IM/DM 数据，以及从合格事件记录可读结果。图 9 展示存储连接和观察状态，后续接口表区分当前事件与稳定停止现场。

observe_wb_valid 表示存在 WB 标签；排除 wb_fault 后才计合法完成，检查 observe_rf_we 后才计 RF 写入。STR、分支和 HALT 因此可以完成而不写 RF。系统用这一事件区分维护 RF 写回影子、最后完成 PC 和最后 store。

4.2.1 文件初始化与系统连接

cpu_system 把核心与 256 字同步 IM/DM 封装为共同仿真和综合顶层。IM_INIT_FILE 和 DM_INIT_FILE 分别指定启动镜像，配置或仿真启动时加载；同步复位保留 DM。all_ops 镜像覆盖 14 种合法 Opcode，系统 TB 通过相同文件启动路径运行。

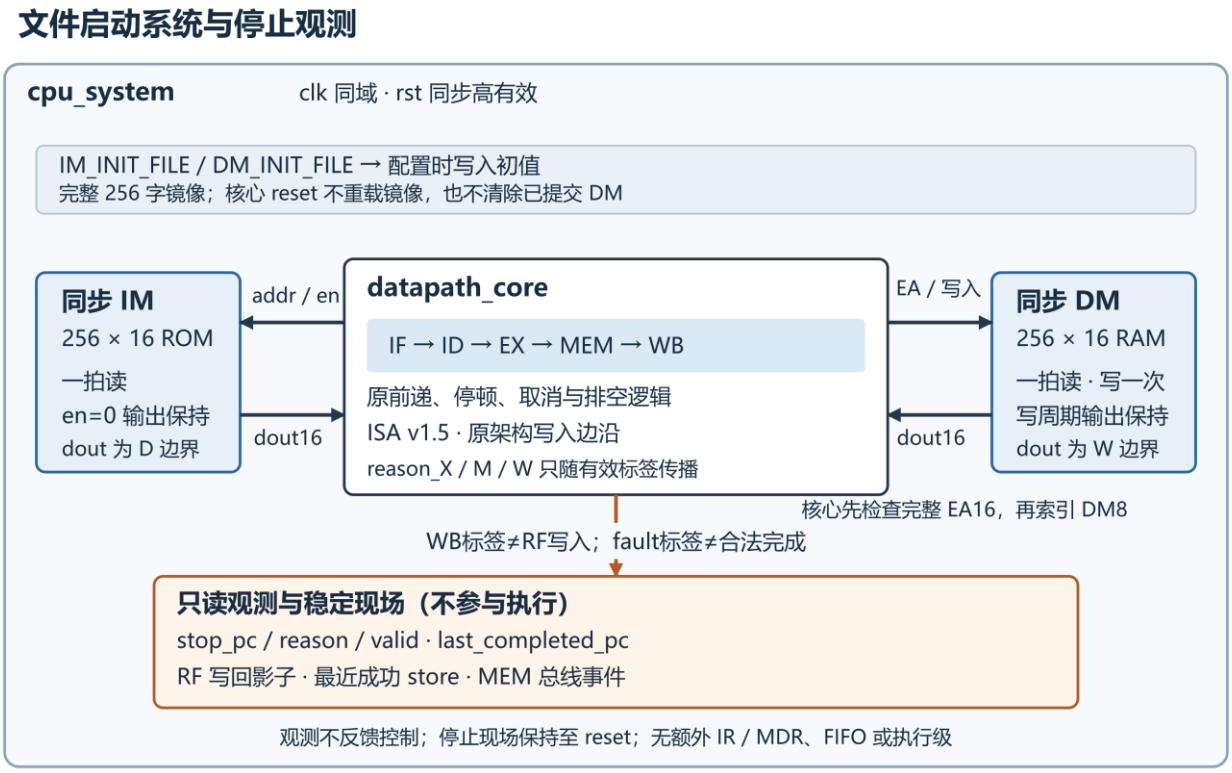


图 9 文件启动系统与停止观测。IM/DM 输出继续承担原 D/W 数据边界；框图不新增流水级。观测区域包含 core 的停止锁存，以及系统顶层中的 RF 写回影子、最后 store 记录。

镜像采用 256 行四位十六进制格式。运行入口在启动工具前检查字数、格式及输入指纹，更换镜像则建立新的独立执行记录。完整地址由核心检查后再接物理 8 位索引；板级复位连接见第 4.3 节。具体命令和记录位置见附录。

4.2.2 稳定停止与公开结果接口

core 另提供 107 位只读观测输出，系统据合格写事件维护 RF 写回影子和最后 store。影子用于查看写回结果，不参与执行取数、前递或控制；信号仅在 valid 或相应写使能有效时解释。

表 35 系统公开观测接口

信号组	位宽与采样规则
WB 事件	valid/fault/halt/store 各 1；rd2、pc16、data16、候选 Z/C/V3；采样上升沿前的合格事件
写入与流水	rf/z/cv_we 各 1；stall/redirect 各 1；fetch_pc16、{D,X,M,W} valid4、架构{Z,C,V}3
稳定停止	stop_valid/stop_pc_valid 各 1、stop_pc16、stop_reason3；原 drain_done 边沿后保持至 reset
完成标签	last_completed_valid1、last_completed_pc16；只记录无故障合格 WB，含 STR 和 HALT
系统结果	rf_snapshot64={R3,R2,R1,R0}；MEM en/we 各 1、addr/data 各 16；last_store_valid1、addr/data 各 16

表 36 停止原因编码

原因	含义	PC 规则
0	尚无停止现场	stop_valid=0
1	HALT	WB 中 HALT 自身 pc
2	非法 Opcode B/F	故障指令 pc
3	取指越界	越界取指标签 pc
4	数据 EA 越界	LDR/STR 指令 pc
5	内部控制非法或消费未就绪源	对应 EX 故障 pc
6	非法二进制 mode 的既有 fail-safe	stop_pc_valid=0，无指令 PC

3 位 reason 随 X→M→W 标签传递。正常或指令故障停止时锁存 w_pc，非法模式则标记无有效指令 PC。现场保持到复位；被取消的标签不产生停止现场。原因 5/6 的证据来自受控仿真注入，详见验证范围。

4.3 板级时钟复位与观测边界

EES-331 实现目标为 xc7z020clg484-1。板级将 M19 的 100MHz 时钟送入 BUFG，PL 按键 S1 位于 L18 并采用低有效输入；VIO 与应用逻辑使用同一时钟。图 10 展示从系统封装到器件顶层的连接。

4.3.1 时钟复位与观测连接

ees331_top 的外部逻辑 IO 为时钟、S1 和八个 LED，相关 PL Bank 使用 3.3V。LED 为高电平点亮；VIO 通过 JTAG 读取内部状态，在 CPU 停止时保持工作。VIO 的独立复位输出与 S1 共同形成复位请求。

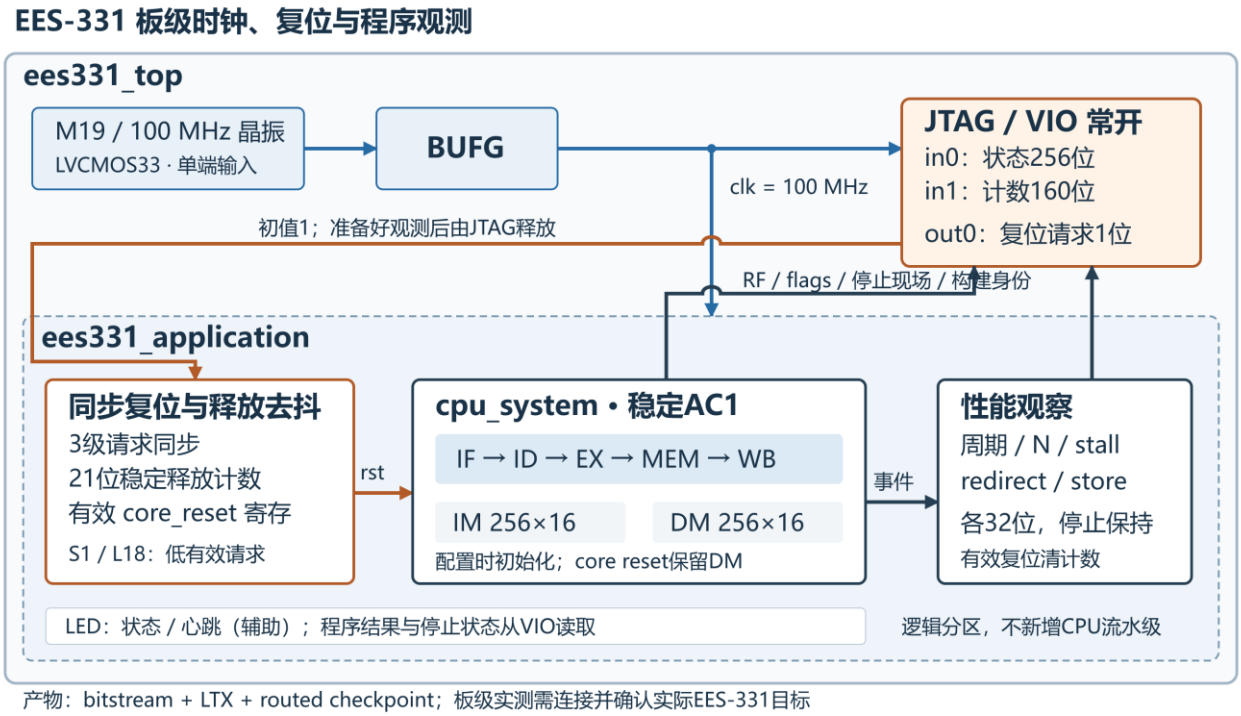


图 10 EES-331 板级结构。虚线内为同一 application 的逻辑分区；不新增 CPU 流水级。VIO 不受 CPU 复位，能够在停止时读取完整现场。

为将异步请求转换为核心同步复位，板级先使用三级同步器，再寄存 core_reset。请求传递之前已完成的合法写入保留；有效复位清 CPU、观察状态和计数，DM 内容保持。请求撤销后需稳定 2^21 周期，按 100MHz 约为 20.97ms，随后释放复位启动 CPU。该等待不计入程序执行周期；RAM 使能由同步逻辑驱动。

4.3.2 完整实现与时序覆盖

五个程序采用一致的 Vivado 2025.2、器件、RAM、VIO 和布局布线配置。完整实现检查 setup、hold、脉宽、厂商调试总线偏斜、内部路径覆盖及十个外部 IO；用户 false-path 只用于异步按钮到首级同步 D，以及无同步接收器的 LED 输出。

DRC 按报告对象解释：PDCN-1569/LUTAR-1 属于厂商 debug hub，RTSTAT-10 表示无负载，ZPS7-1 对应 PL/JTAG 启动，SYNTH-6 对应固定一拍 IM/DM 且未增加输出流水级。相关告警保持原等级，CPU 受影响路径仍须单独检查。功能和实际板级结果见第 4.4 节，资源及时序数值见第 4.5 节。

4.4 分层验证方法与证据范围

验证按单元、流水控制、文件启动系统和实板逐层集成。单元与 ISA 模型确定预期，流水断言检查发生时刻，系统程序连接真实同步存储接口，最终通过 EES-331 稳定终态和计数核对默认 AC1。

下述结果保留各自的实际执行身份，第三章五个实例来自同一批已绑定当前默认源码的逐拍 CSV。运行来源集中列于附录，正文按方法和覆盖对象组织。

4.4.1 自检查测试方法

验证先建立独立预期，再比较架构事件与逐拍推进。ISA 参考模型按指令语义计算 RF、DM 和标志；TB 在边沿前采样使能，在非阻塞赋值更新后检查状态。关键控制使用$isunknown 和!==捕获 X/Z，无效且未消费的数据允许保持旧值。以下为单元 ALU 检查节选。

```
expected_c=0; expected_v=0; expected_bad=0;
case (f)
  0: expected_result=y;
  1: arithmetic_oracle(x,y,0,expected_result,expected_c,expected_v);
  2: arithmetic_oracle(x,y,1,expected_result,expected_c,expected_v);
  3: expected_result=x&y;
  4: expected_result=x|y;
  default: begin expected_result=0; expected_bad=1; end
endcase
#1;
if ({illegal,alu_c,alu_v,alu_result} !==
    {expected_bad,expected_c,expected_v,expected_result} || eq !== (x==y))
  $fatal(1, "ALU mismatch");
```

算术参考模型先计算精确整数结果，再截取低 16 位。ADD 用扩宽和判断 C；SUB 直接比较无符号 a≥b 判断无借位。V 先将输入解释为 16 位补码，再扩展到 32 位整数计算，判断结果是否超出−32768 至 32767。表 13 的手算边界向量用于交叉检查该模型；#1 仅为仿真检查等待。

CLA4 另检查 P/G 及其对 cin 的独立性，CLA16 另检查组间超前 K1 至 K4。AC1 运行路径则检查 result、无借位 C、V 和 eq。单元检查、逐拍断言与架构结果比较共同组成验证链，各自的覆盖范围如下。

4.4.2 单元与流水覆盖总览

表 37 AC1 已存实际运行范围

对象	实际检查数量	结果与边界
译码/边界	65,536 字及 14 手算 ALU 例	PASS；ISA v1.5 不变
原主回归	128 程序+16 复位，3,380 有效完成	PASS；128 最终状态和周期相同

对象	实际检查数量	结果与边界
原单元	CLA4 512；CLA16 与 addsub 各 564,300；ALU 2,257,200；扩展 512；flags512；RF262,161；PC65,539；目标 3,584	PASS；CLA 为独立单元；各输入空间见表 38
原压力	4,696 案例，33,134 有效完成，1,408 fault tokens	PASS；加统一 WB 数量与 M/W 标签检查
原独立手写	12 定向+4 重启	PASS；旧架构状态预期保留
RAW 矩阵	1,728 单源+648 同源双读=2,376	PASS；9 生产者、10 消费者、4 寄存器、3 距离
假相关/Z/交叉	68 / 84 / 22；另复跑 16 复位	PASS；新 hazards 总计 2,550
前递控制空间	2^17=131,072；4 个不同数据哨兵	PASS；非完整 16 位数据穷举
统一 WB smoke	5 完成；1 DM 写；1 STR WB；1 branch WB	PASS；v1.5 同测试失败属历史
短时序样例	12 周期，5 完成，1stall，1redirect	PASS；原生 interlock.vcd 保留

表 38 单元模块的精确覆盖范围

对象	输入或状态范围	验收重点
CLA4	a4 b4 cin1 共 512 组	sum cout P/G 及 cin 独立性
CLA16 独立单元	a16 b16 cin1 全量为 2^33	结果与进位 长进位 组间超前结构
AC1 ADD/SUB 核心	a16 b16 sub1 全量为 2^33	每种运算 2^32；result/C/V 均检查
ALU	a16 b16 op3 全二进制空间 2^35	5 合法及 3 非法 op eq 独立 非算术 C/V=0
新译码器	全部 65,536 指令字	两张控制表 源/use 选择 扩展与副作用
扩展器	imm8 mode1 共 512 组	零扩展 符号扩展
flags_reg	8 旧状态×2rst×4 使能×8 输入 共 512 组	复位 保持 独立字段及同时写

表 38 列出精确输入空间，表 37 列出已有运行完成数。CLA4、译码器、立即数扩展和 flags 在列明空间内完成全量检查；16 位加减与完整 ALU 仍是有限输入集合。单 op 的 2^32、带 cin/sub 的 2^33 和全 ALU 的 2^35 分开记录，长回归只有取得最终完成日志后才可标为穷举通过。

4.4.3 流水时序与指令执行检查

主 TB 逐边沿检查有效位和 pc 推进、停顿时 PC/D/IM 保持、EX bubble 及无效槽副作用。WB 完成检查与 MEM 写事件分开，防止 STR 重复计数或提前丢失完成标签；多生产者情况下另检查未就绪 load 对旧同名值的阻断。

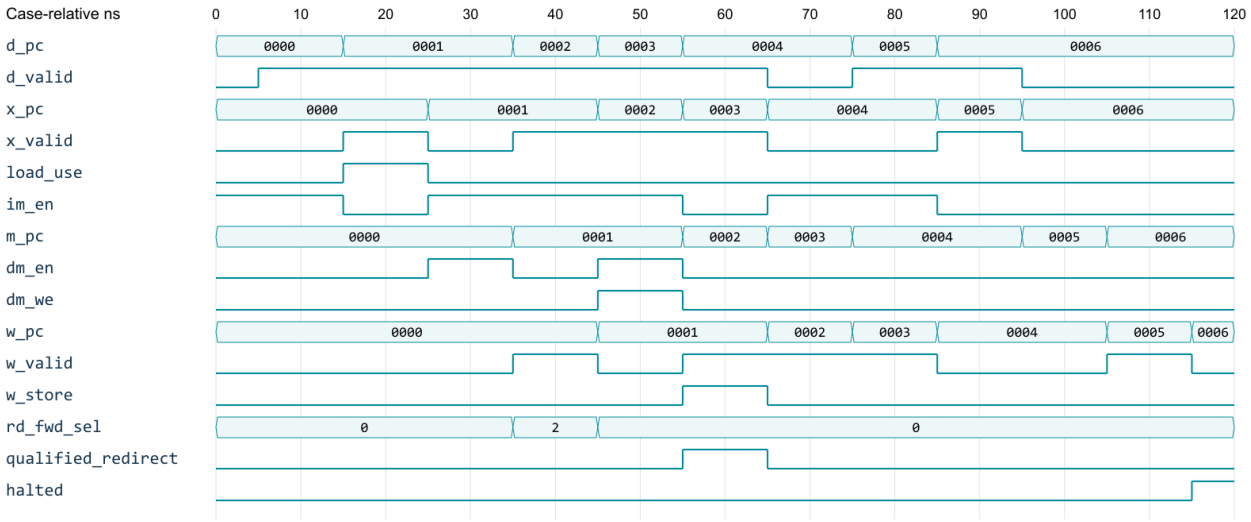


图 11 冻结 v1.6 的 load 到 STR 波形。C3 保持、C4 插 bubble、C5 从 W 前递，STR 继续保留有效 WB 标签。

样例机器字 8300 9100 A000 D001 F000 E000；R1=0021，R3=0020，DM[0020]=0007。C3 的 load_use 禁止取指并保持 ID；C5 的 STR 从 WB load 取得 0007，C6 在 MEM 写 DM[0021]，C7 仍有有效 STR WB 且无写入。JZ 取消年轻 F000，C12 HALT WB 结束后停止。横轴为用例相对 ns，Ck 结束边沿在 5+10×(k−1)ns；不要把同列 D 机器字和 X pc 当作同一条指令。

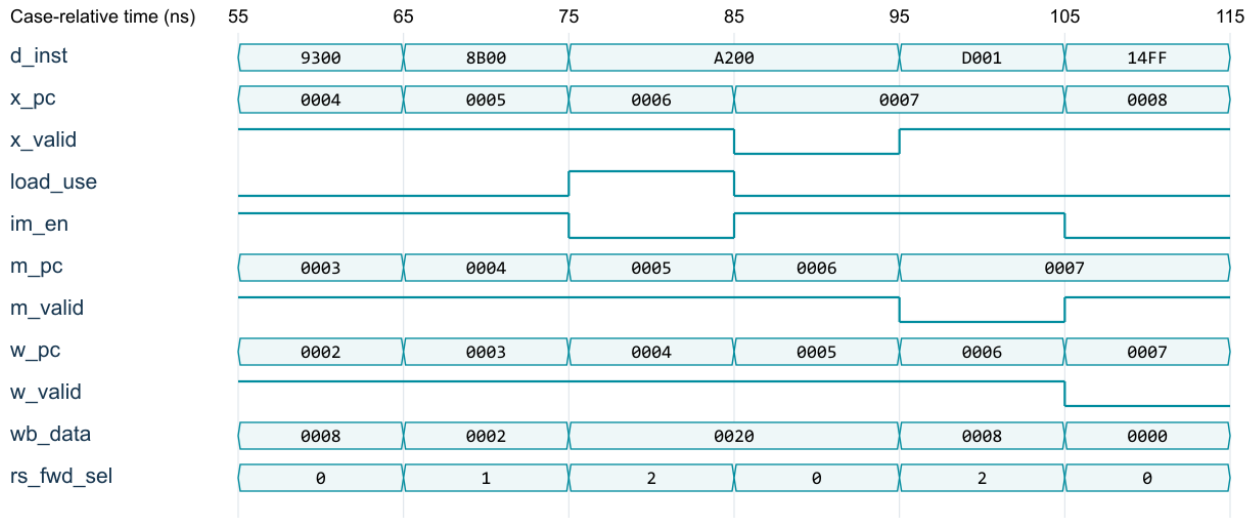


图 12 冻结 v1.6 基准 load-use 波形。AC1 对应逐拍 CSV 与该周期契约一致；指令按各级 pc/valid 定位。

表 39 v1.6 基准波形与逐拍记录的对应关系

周期位置	有效载荷或条件	观测结果	检查的行为
C9 边沿前	X 的 LDR pc6，D 的 CMP pc7	stall=1，im_en=0；M 的 STR pc5 有效	只保持前端，老 STR 在本边沿写 DM[0020]=0008
C10 边沿前	M 的 LDR pc6，X 为 bubble	dm_en=1 dm_we=0；DM 读在结束边沿返回	load 与 WB 标签同边沿对齐；老 STR 为有效 WB，无架构写入
C11 边沿前	W 的 LDR pc6，X 的 CMP pc7	W 返回 0008 并转发至 CMP 实际源	不将 M 级 load 地址当作数据
C12 边沿前	X 的 JZ pc8，M 的 CMP pc7	redirect=1；目标 000A；候选 Z=1	最近 CMP 前递，取消年轻 pc9 及 IF 响应
C15 至 C17	HALT pcA 依次占 EX MEM WB	C17 结束边沿 halted=1，fault=0	WB 消费停止标记；10 条动态指令 17 周期

VCD 记录各级 pc/valid、转发选择、地址和 store 数据、WB 资格以及候选和保存的 Z/C/V。图 12 选取 baseline 加载相关窗口，表 39 将波形位置与边沿动作对应；其他案例结合各自 CSV 和日志定位。

读图时依次确认输入稳定、组合结果、边沿锁存和架构写入，并用 pc/valid 绑定同一条指令。CU 逐拍检查同时观察请求与合格事件，确认因年龄被拒绝的请求没有副作用。

4.4.3.1 流水相关与停止测试

表 40 的定向测试覆盖标志保持、连续 CMP、访存依赖及停止交叉事件。各例从独立复位开始，TB 在第一条取指前设置列明 RF 初值；未列 RF、Z/C/V 和 DM 为 0，连续机器字从 IM 地址 0000 装入。表中预期与已有 CASE_PASS 及逐拍见证对应。

表 40 流水相关与停止定向用例

用例	初始条件与机器字序列	预期状态与时序
T01 CMP 后算术	R0=0001 R1=0001 R2=FFFF；A100 5801 D001 1C55 E000	ADD 后 R2=0000 C=1 V=0；Z 仍 1；JZ 跳过 MOV R3
T02 连续 CMP	R0=0001 R1=0002；A000 A100 D001 1855 E000	最近 CMP 输出 0 覆盖旧 1；JZ 不跳；R2=0055
T03 地址运算保持标志	R0=7FFF R1=0020 DM[001F]=BEEF；5001 89FF 9900 E000	R0=8000 C=0 V=1；R2 及 DM[0020]=BEEF；访存保持 C/V；load 到 STR 数据停 1 拍
T04 错误路径写入	R0=FFFF；C003 5001 A000 E000	ADD/CMP 均被取消；RF 和 Z/C/V 保持
T05 老 WB 与年轻故障	R0=7FFF R1=00FF；5001 8901 5801 E000	ADD 在 WB 与 LDR 在 MEM 越界同拍；R0=8000 C=0 V=1；故障 load 和年轻 ADD 无写入
T06 load use 与 CMP	R1=0020 R2=0001 DM[0020]=0001；8100 A200 D001 1C55 E000	load 到 CMP 停 1 拍；R0=1 Z=1；JZ 跳过 MOV R3
T07 HALT 排空	R0=0020 DM[0020]=1234；8400 E000 9401	老 LDR 写 R1=1234；HALT 后 STR 取消，DM[0021]保持 0
T08 停顿不冻结 WB	R0=7FFF R1=0020 DM[0020]=0001；5001 1C00 8900 4A00 E000	load 到 ADD R2,R2 停 1 拍；同拍老 ADD 写 C=0 V=1；末尾 R2=2 C=0 V=0

T09 对应 tb_datapath.sv 的 reset_probe，共 16 次。正常序列为 1020 1405 9400 8800 A200 E000，以 phase=0 至 12 调用；故障序列为 1001 B000 E000，以 phase=4 至 6 调用。TB 在下降沿断言 rst 并跨越下一上升沿，检查 RF、Z/C/V、PC、valid 和停止状态清 0，DM 全部保持。正常序列 phase=5 使待写 STR 遇到复位而禁止写入，phase=6 确认已经完成的 STR 不被清除；故障序列覆盖 DRAIN_FAULT 和 FAULTED 复位。随后另一次用例准备再撤销 rst；独立 audit3 另有 4 次停止边沿或停止后复位并重启的测试。T09 是时序测试，不能用顺序指令模型替代。

定向检查同时核对最终状态和必要时序见证：T01 检查算术后 Z 保持，T05 检查 MEM 故障同拍老 WB 完成，T08 检查前端停顿同拍老 WB 完成。缺少要求的时序见证即失败。

flags_reg 的 512 组覆盖采用“建立旧状态，再执行一次计数转移”的方法，建立状态的时钟不计入 512。CU 测试覆盖图 8 的可达转移、RUN 下停顿/重定向、DRAIN 期间较老写入、停止保持和各状态复位。T07 断言 HALT 在 MEM 时 halted 仍为 0，在有效 HALT 的 WB 结束边沿置 1，随后所有 valid 为 0 且停止期间没有新访问。T16 检查 MEM 故障与 HALT 同拍时直接进入 DRAIN_FAULT，不应经过 DRAIN_HALT 或 HALTED。DRAIN_HALT 中不得出现新的已确认故障，否则 TB 报告时序契约被违反。故障测试对 FAULTED 采用相同的 WB 标记断言。测试设超时退出，避免错误控制导致无限等待。

T17a/T17b 在故障 STR 的 MEM 周期检查完整 EA、dm_en=0 和 dm_we=0；逐字比较 DM，确认没有按低 8 位误写 0000 或 00FF，同时检查老 WB 完成及年轻操作取消。该故障 STR 在下一拍保留有效 WB 故障标记，结束边沿进入 FAULTED。T18a 对越界取指检查 im_en=0、IF/ID 的 pc=0100 和 fetch_fault=1；故障沿有效载荷传到 EX 确认，再于 WB 完成停止。T18b 检查更老分支取消年轻取指故障，目标 HALT 正常结束。状态断言在采样边沿的非阻塞赋值更新完成后执行；DM/IM 使能在边沿前采样。

4.4.3.2 全部指令的执行覆盖

表 41 补齐 MOV 寄存器、SUB 寄存器、AND、OR、保留编码故障和负位移分支。每个子例独立复位，沿用上述初值规则；正常程序未使用的 IM 字填 B000，稀疏镜像按表中地址装入。

表 41 指令执行与故障定向用例

用例	初始条件与机器字序列	预期状态与时序
T10 MOV 寄存器	R0=55AA；Z=C=V=1；140F 0100 E000	R1 及 R0=000F；Z/C/V 保持 1；紧邻 MOV 使用 Rs 转发，不读取旧 Rd
T11a SUB 无借位及溢出	R0=8000 R1=0001 Z=1；6100 E000	R0=7FFF C=1 V=1 Z=1；R1 保持 0001
T11b SUB 借位	R0=0000 R1=0001 Z=1；6100 E000	R0=FFFF C=0 V=0 Z=1；不能把 borrow 直接当作 C
T12 AND	R0=A5A5 R1=0F0F；Z=C=V=1；2100 E000	R0=0505 R1=0F0F；Z/C/V 保持 1，不被非算术 C/V 候选 0 覆盖
T13 OR	R0=A500 R1=00A5；Z=C=V=1；3100 E000	R0=A5A5 R1=00A5；Z/C/V 保持 1
T14a 保留 B 编码	1001 B000 1402 E000	老 MOV 完成，R0=0001；年轻 MOV/HALT 取消；R1=0 fault=1 halted=0
T14b 保留 F 编码	1001 F000 1402 E000	与 T14a 相同；故障标记在 WB 消费前 fault 仍为 0

用例	初始条件与机器字序列	预期状态与时序
T15 JZ 负位移	1002 1401 7001 A100 D0FD E000	pc=0004 的目标为 0002；第一次跳，第二次不跳；末尾 R0=0 R1=1 Z=0 C=1 V=0
T16 MEM 故障优先于 HALT	R1=00FF R2=55AA；8901 E000 1001	LDR 越界同拍取消 EX 的 HALT；R2 保持 55AA R0=0；fault=1 halted=0
T17a STR 上界越界	R0=7FFF R1=00FF Z=1 DM[0000]=CAFE DM[00FF]=BEEF；5001 9101 1C55 E000	EA=0100；DM 全部保持；老 ADD 写 R0=8000 C=0 V=1 Z=1；年轻 MOV 取消；fault=1 halted=0
T17b STR 负位移越界	R1=0000 DM[00FF]=BEEF；1001 91FF E000	EA=FFFF；DM 全部保持；老 MOV 写 R0=0001；HALT 取消；fault=1 halted=0
T18a 正确路径取指越界	IM[0000]=C0FF IM[00FF]=1001；其余 IM 填 B000	跳至 00FF，MOV 写 R0=0001；下一取指 0100 产生故障；fault=1 halted=0；DM 保持
T18b 分支取消取指故障	IM[0000]=C0FF IM[00FF]=C002 IM[0002]=E000；其余 IM 填 B000	00FF 的 JUMP 取消年轻 0100 取指故障，转至 0002；HALT 正常结束；RF/DM 保持 0，fault=0 halted=1

T18a/T18b 使用稀疏 IM 地址。T18b 的关键见证是年轻 0100 取指故障被更老分支取消，未进入有效 EX；目标 HALT 正常完成。其余连续序列按 0000 起装入。

表 42 把 14 种合法 Opcode 映射到实际 CPU 执行用例，检查从译码、数据源选择到规定副作用的完整路径；批量回归再改变寄存器索引、立即数边界、RAW 距离与初始标志。

表 42 全部合法指令的 CPU 执行覆盖

指令编码	指令形式	CPU 执行用例	必须检查
0	MOV Rd,Rs	T10	PASS_B 与 Rs 选择；Rd 写回；标志保持
1	MOV Rd,#u8	T10 及功能程序	零扩展；不使用旧 Rd；标志保持
2	AND	T12	按位结果；只写 RF；Z/C/V 保持
3	OR	T13	按位结果；只写 RF；Z/C/V 保持
4	ADD Rd,Rs	T08 及功能程序	两源转发；RF 与 C/V 写回；Z 保持
5	ADD Rd,#u8	T01 T03 T08	零扩展；C/V 边界及 Z 保持

指令编码	指令形式	CPU 执行用例	必须检查
6	SUB Rd,Rs	T11a T11b	C 为无借位；V 独立；Z 保持
7	SUB Rd,#u8	T15 及功能程序	零扩展；低 16 位结果与 C/V
8	LDR	T03 T05 T06 T07	符号位移；DM 读及 WB；越界禁止写回
9	STR	T03 及功能程序；T17a/T17b	合法写入；越界禁止 DM 访问；故障标记保留
A	CMP	T01 T02 T06	只写 Z；最近 CMP 优先；不写 RF/C/V
C	JUMP #u8	T04 T18a T18b	绝对目标；取指边界；年轻副作用取消
D	JZ #s8	T01 T02 T15	跳与不跳；负位移；使用指令 pc 标签
E	HALT	T07 及功能程序	老操作完成；年轻操作取消；WB 结束标记

B/F 不是合法指令，分别由 T14a/T14b 检查正确路径故障；功能程序另检查 HALT 取消年轻保留编码。覆盖计数只在有效指令完成其规定动作时更新，取到或译码到一条最终被取消的指令不算该指令已执行。除目标 RF/DM/标志外，TB 还要检查未指定写入的寄存器、标志及 DM 保持不变，并对每次实际副作用核对 pc 标签、目的和数据。

4.4.4 文件启动整机验证

下面程序从正常复位状态运行，使用 MOV 建立输入，不依赖 TB 修改 RF 内部状态。它覆盖相邻算术相关、存储与加载、load 到 CMP、Z 转发、错误路径取消及 HALT 结束。

表 43 CPU 功能与性能基准程序

字地址	机器字	指令	预期动作
0000	1005	MOV R0,#5	R0=0005
0001	1403	MOV R1,#3	R1=0003
0002	4100	ADD R0,R1	转发源值；R0=0008 C=0 V=0
0003	7401	SUB R1,#1	R1=0002 C=1 V=0
0004	1C20	MOV R3,#32	R3=0020
0005	9300	STR R0,[R3+0]	DM[0020]=0008
0006	8B00	LDR R2,[R3+0]	R2=0008
0007	A200	CMP R0,R2	load use 停 1 拍；Z=1
0008	D001	JZ +1	目标 000A；使用 CMP 候选 Z

字地址	机器字	指令	预期动作
0009	14FF	MOV R1,#255	错误路径；不得执行
000A	E000	HALT	排空后 halted=1

baseline 的实际终态与预期一致：R0=0008、R1=0002、R2=0008、R3=0020、DM[0020]=0008、Z/C/V=1/1/0、fault=0。C=1 来自 SUB 的 3−1 无借位，之后各指令保持 C/V；CMP 更新 Z。11 个静态机器字中有 10 条动态完成指令。

本测试的其余 IM 字填 B000，DM 初始填 0。HALT 到 EX 确认前可能已取到其后的保留编码；这些年轻故障标记应被 HALT 取消，不能一到 ID 就设置全局 fault。停止程序的正确结果仍为 halted=1、fault=0。

表 44 文件启动系统与已存工具验证

检查	真实执行结果与范围
独立用例与回归	复用表 37 的有限回归，五类错误注入均触发预期检查；X/Z 及结果记录检查单独保存
同步存储	7 项公开端口检查：文件读、逐拍更新、禁用保持、写时保持与写后读
系统程序	11 个程序、12 行完整 epoch 结果；全合法 Opcode、控制/访存故障、年轻取消、HALT 前老访存、两类 reset 与停止保持
周期与网表	baseline=17/10（周期/动态指令），1 停顿 1 跳转；all_ops=25/16，1 停顿 2 跳转；all_ops 网表回放完全同拍
工程启动	实际创建 14 份 RTL 的 Vivado 工程，并经 launch_simulation 完成相同 11 程序检查
系统 OOC	Vivado 2025.2；xc7z020clg484-1；10ns；系统 OOC 资源及时序见表 46

系统仿真通过文件启动验证正常和故障程序、复位及稳定停止。all_ops 的综合网表回放保持同拍行为。系统 OOC 包含双存储和结果影子，其资源边界与完整板级实现分开记录，见第 4.5 节。

4.4.5 EES331 实板功能验证

五个默认 AC1 已布线镜像分别在 EES-331 运行。每个程序读取两份原始 VIO 快照，核对构建身份、稳定终态和计数；随后在 checksum 正常停止后实际操作 S1，检查复位清零与松开后重启。

表 45 EES-331 实板功能与复位结果

检查对象	实际记录	可支持的结论
五个正常 HALT 程序	all_ops、baseline、loop、memory_copy、copy_checksum；各两份原始 VIO 快照	RF 写回影子、Z/C/V、停止 PC、最后 store 及计数符合独立预期
周期计数	25、17、30、15、55	默认 AC1 五程序计数与周期模型一致
停止后 S1 复位	按住时 core_reset=1，公开状态与计数清 0；松开后 checksum 重获 55 周期终态	已停止 CPU 可通过实际 S1 复位重启
证据范围	稳定终态和计数来自实板；内部逐拍和故障案例来自仿真	独立频率、直接运行时间、功耗及实验候选的实板验证尚未完成

RF 观测值来自写回影子。板上稳定快照与仿真内部逐拍记录分别对应终态、计数和控制时序；原始 VIO、S1 及仿真记录的来源见附录。

4.5 性能基线与统计口径

性能先按同一工作负载统计周期，再结合实现边界比较面积和时序。核心、文件启动系统 OOC、含 VIO 板级分别计量，候选与 AC1 采用相同程序、RAM、器件、调试配置和工具约束成对比较。

五个默认 AC1 程序的周期来自实板 VIO。按 100MHz 设计频率可换算程序时间；该换算与独立仪器直接计时分开解释。

4.5.1 CPU 性能验证

C 从第一个有效 IF 周期开始，计至 HALT 停止标签被 WB 消费的边沿，包含流水填充、停顿和冲刷，排除复位准备。N 统计合法完成的动态指令，包含 STR、CMP、分支与 HALT，排除 bubble、错误路径和故障 token。STR 的 MEM 写入与下一拍 WB 完成分别观察，N 只计后者一次。

```
CPI = C / N
IPC = N / C
T_program = C / f
MIPS = f / (CPI * 1000000.0)     // f in Hz
```

对本节五个正常 HALT 程序，周期分解为 C=N+4+S+2R：4 为固定流水开销，S 为已执行的数据停顿，R 为 EX 合格重定向次数。该关系对应本设计的固定一拍存储和无额外等待条件。故障或复位程序按实际逐拍记录计数。

TB 统计 C/N 时保留足够位宽的整数计数器，计算 CPI、IPC 和程序时间时转为实数，避免整数除法截断。上述是度量关系，不是需要在 CPU 中综合的除法电路。

报告分别给出 C/N、load use 停顿数、取分支次数、实际工作时钟以及 LUT/FF/BRAM/CARRY 资源。程序时间 T 是比较结果，CPI 和 f 应一起说明；综合资源或理想 IPC=1 不等于 CPU 性能验证完成。Fmax 只在约束、布局布线和完整 setup/hold 检查范围内讨论，行为仿真的#1 延迟不用于推导 Fmax。

表 46 三种资源和时序统计边界

对象	LUT / FF / BRAM18	时序与适用范围
历史 core-only AC1	287 / 260 / 0	历史 OOC WNS +2.620ns；观测扩展前，不能当整机资源
文件启动 cpu_system	376 / 431 / 2	已存系统 OOC WNS +1.363ns；含 IM/DM 和影子
含 VIO 板级 AC1	五程序 LUT1082 ~ 1160；FF2822 ~ 2826；BRAM18 均 2	完整布线 setup WNS +1.434 ~ +2.101ns；不是实物时钟或时间测量

两项 OOC 均使用 15 个 CARRY4。系统 OOC 的 TNS 为 0.000ns、失败端点为 0；完整板级的 setup 和 hold 分别按下表布线报告读取。

4.5.2 固定程序与性能基线

表 47 五个工作负载的完整实现基线

工作负载	周期 / N	stall / 跳转	LUT	FF	setup / hold ns
all_ops	25 / 16	1 / 2	1118	2826	2.086 / 0.071
baseline	17 / 10	1 / 1	1120	2825	1.838 / 0.083
loop	30 / 18	0 / 4	1160	2822	2.101 / 0.062
memory_copy	15 / 7	2 / 1	1119	2823	1.540 / 0.086
copy_checksum	55 / 35	16 / 0	1082	2824	1.434 / 0.073

五个配置都使用 2 个 BRAM18；面积采用 routed utilization 的 Total LUTs，包含 LUTRAM。每个程序的 98 项共同输入相同，选定镜像和独立 gold 数据是明确变量。程序不同可能导致常量优化及资源差异，候选比较必须使用同一程序成对评价。

复制校验程序包含 35 条动态指令：8 次 load/store 复制，再读回 8 个目的字并累加。初值 FFFF、0001、8000、7FFF、1234、A5A5、5A5A、0000，最终 R1=1232，R0/R2=0000，R3=0020；最后 store 地址 0047、数据 0 且 valid=1。AC1 实际为 55 周期、16 停顿，避免仅用最后一个复制字证明整段程序。

按已实现 10ns 时钟计算，五个程序时间为 250、170、300、150、550ns。周期数已由实板 VIO 核对；这些纳秒数仍按 100MHz 设计频率推算，不是独立仪器计时，也不含复位去抖、配置和 JTAG 等待。实际 CPU 时钟必须与布局布线配置和板卡时钟相符。

4.6 隔离性能候选与取舍

两个候选分别检验组合 MUX 改写和延迟消费 STR 数据的价值。前者应保持每拍行为；后者调整特定 STR 相关的停顿规则，因此同时检查 ISA 事件顺序与新周期契约。两个候选独立评价，默认核心继续采用 AC1。

正常 HALT 的 copy_checksum 含 35 条完成指令。AC1 为 35+4+16=55 拍，WB 到 MEM 候选为 35+4+8=47 拍；少的 8 拍来自 STR 数据相关等待。以下分别说明机制、覆盖、实现代价及保留候选的原因。

4.6.1 前递 MUX 组合候选

动机是改善前递 MUX 的组合实现。forward_onehot 将数据选择改为掩码 OR，保持相关比较、sel 和 blocked 的优先级，M load 仍阻断旧 W。已有九套、独立 48 例、错误注入及五程序公开逐拍轨迹检查通过；四态 Z 数据的完整等价性超出这组有限检查。

表 48 前递 MUX 候选相对 AC1

程序	LUT 变化	setup 裕量变化 ns	100MHz 周期变化
all_ops	21	+0.136	0
baseline	13	-0.148	0
loop	-20	-0.488	0
memory_copy	15	-0.265	0
copy_checksum	108	-0.426	0

该候选在四个工作负载增加 LUT，在四个工作负载降低 setup 裕量，100MHz 下周期不变。因此组合表达式改写未形成稳定收益，默认实现保留 AC1。

4.6.2 WB 到 MEM 存储数据候选

相邻 LDR→STR 只有写入数据相关时，STR 真正消费数据是在 MEM。wb_mem 因此将这项消费延后：EX 保留占位数据和 m_store_pending，复用 m_rd 记录源索引；MEM 再检查 WB 中同名、有效、无故障、可写的 load，选择已就绪 wb_data。若基址 Rs 也相关，EX 仍需要一拍停顿。

EX 例外必须匹配合法 STR 控制；未知或非法资格不能绕过 blocked 检查。MEM 未就绪时禁止写 DM 并发内部故障原因 5，地址越界保持原因 4；较老 MEM 事件优先年轻 EX，老 WB 继续完成。STR 在 MEM 写一次，其有效标签继续 WB，无 RW/ZW/CVW。reset、取消和 bubble 均不能保留可产生副作用的 pending。

表 49 WB 到 MEM 候选的实际周期比较

程序	AC1 周期 / stall	WB 候选周期 / stall	周期推算差值 ns
all_ops	25 / 1	25 / 1	0
baseline	17 / 1	17 / 1	0
loop	30 / 0	30 / 0	0
memory_copy	15 / 2	14 / 1	-10
copy_checksum	55 / 16	47 / 8	-80

该候选已通过 48 个原 ISA 用例和 10 个真实复位见证，以及 2005 个候选用例（2002 个正常 HALT、3 个故障，其中 64 个为固定种子压力程序）。逐边沿检查 RF、DM、标志、指令顺序、STR 到 WB 标签和停止状态；正常 HALT 程序另检查新周期与停顿数。另有 12 项内部协议/复位保护检查和 6 个被明确捕获的错误变体，包括非法和 X 控制不能借用 EX 延迟数据例外。此为有限仿真，不等于全状态形式证明。

原冻结 48 例保持原字节；私有执行 TB 仅排除两条旧版总周期/停顿期待，架构断言、复位见证及新控制检查继续生效。新候选不要求旧时序 CSV 逐字节相同，而要求架构事件与候选周期契约同时成立。故障/reset 不机械套正常 HALT 公式：年轻 STR 减少停顿，不一定缩短较老 fault 自身排空时间。

表 50 WB 到 MEM 候选的完整实现代价

程序	LUT 变化	FF 变化	setup 裕量变化 ns
all_ops	-2	2	-0.303
baseline	85	2	-0.099
loop	-50	2	-0.566
memory_copy	84	2	-0.047
copy_checksum	86	2	+0.234

WB 到 MEM 可减少存储数据相关负载的周期，但增加资格检查、pending 状态和数据选择。copy_checksum 少 8 拍并增加 86 LUT、2 FF，memory_copy 少 1 拍并增加 84 LUT、2 FF；完整代价见表 50。由于尚无候选实板验收，保留为隔离实验，默认 AC1 不变。时间差按固定 100MHz 换算。

附录A　版本演进与证据索引

A.1 历史编码迁移与示例勘误

表 51 当前 ISA 与 Pre4 已验证映射的对应关系

指令	本版 v1.5 Opcode	Pre4 v1.4 Opcode	迁移要求
AND	2	8	两实际寄存器源，alu_op=3；只写 RF
OR	3	9	两实际寄存器源，alu_op=4；只写 RF
LDR	8	2	Rs 加符号位移；MR/RW=1，wb_sel=1
STR	9	3	独立基址与数据源；MW=1，RW=0

其余 12 个 Opcode 的含义不变，包括 B/F 保留编码。新旧机器字不能混用：v1.4 的 3300 是 STR，本版则是 OR；v1.4 的 2B00 是 LDR，本版则是 AND。迁移时重编译指令镜像，并检查译码、参考模型和逐拍观测属于同一版本。

机器字按字段计算并独立校核。确认的编码表右侧有八项示例与字段不符：MOV/AND/OR/ADD/SUB 的 R0,R1 形式以及 CMP R0,R1 都须令 Rs=01，分别为 0100/2100/3100/4100/6100/A100；LDR R0,[R1+2] 为 8102，STR R0,[R1+2]为 9102。这些修正不改变其 Opcode 或汇编语义。原表的 8802/9802 实际选择 Rd=R2、Rs=R0，不能解释成 R0 与 R1。

JZ −2 的 DFFE 不是功能错误：其未使用 Rd/Rs 均为 11，按本规格仍忽略这些字段，offset=FE。程序生成器采用未使用字段为 0 的规范形式 D0FE。最终 PPT 第 5 页 STR R1,[R2+4]的 3604 属于旧映射，本版应为 9604。保存的原 PPT、源码和波形保留历史身份，后续程序镜像仅使用本版统一编码。

A.2 运行身份与历史证据

以下索引用于查找原始运行身份。核心既有记录、Pre5 快照、当前架构图例和板级记录分别保存；重复执行相同测试的结果不叠加为新的覆盖数量。

2026 年 10 月 14 日的 AC1 核心九套回归保留原运行日志；2026 年 10 月 17 日的 Pre5 九套回归绑定 19 次原生调用，attempt 为 440e0d7a-1244-4333-a4f8-5adf612f9833。Pre5 是观测扩展前快照，当前维护入口为 CPU 核心设计。

2026 年 10 月 29 日曾重新解析两份历史 VCD 的 4,824 个上升沿，并核对 11 份控制 CSV/JSON。这是已存证据复核。当前第三章例子采用 r3 实际独立 48 例记录，结果为 48 案例、10 个复位重启见证、58 行结果和 952 行逐拍记录；不是本次编辑新增的仿真。

九套回归的 AC1 原记录 72d99981 及核心包 job1 记录 eae32d52 分别绑定 13 份 RTL、TB、脚本和镜像。九份 CSV 保持原比较关系。完整记录标识以证据索引.json 为准；本规范引用的具体指纹在文档源与证据索引中维护。

阶段 1 至 2 完整工具记录：验证/tools_50040838e0ec474680085bb91e8fb001；系统仿真、综合、网表及工程启动：验证/cpu_system_n0r4buws。第三章 CSV 来源由文档构建证据绑定到验证记录 /cpu_ac1_independent_lh6jb16m。

实板验收记录：验证/hardware_ac1_20261102_3dw9zgb5/result.json。2026 年 11 月 2 日确认 EES-331 实物与单板 JTAG 目标后，使用已布线的默认 AC1 镜像完成五程序及 S1 核对。目标未确认的早期尝试保留原状态，不计为通过。

完整 RAW 矩阵位于说明/Hazard_Matrix_2376.csv，包含生产者、消费者、源角色、间距、机制、预期及实际结果。测试标识 RAW_P*、NEWEST、NEW_LOAD_AFTER_TWO_WRITERS、FALSE_*、FLAG、FLUSH_CLASS_*及 MEM_FAULT_*用于对应第三章机制。

迁移前旧译码器与新 ISA 有 16,384 个二进制指令字不相容，涉及 2/3/8/9。该诊断及 Pre4 原始源码、PPT 和日志保持历史身份；当前译码、镜像和参考模型统一使用第一章 v1.5。

观测扩展前 core-only、文件启动系统、完整板级为不同统计对象。冻结 v1.6 曾记录 WNS=-1.580ns；AC1 历史 core-only 改善为+2.620ns，当前板级结果见第 4.5 节。旧 CLG400 器件截图与 CLG484 课程板封装不同，历史记录保留其原器件。

附录B　复现与维护

B.1　复现入口与文档维护

从仓库根目录运行下列入口，或使用脚本绝对路径。工具选择优先级为显式 CLI、tools.local.json、CPU_VIVADO_BIN/CPU_PYTHON、PATH；个人工具路径保存在本机配置。

每次执行建立新 attempt，保存源码和镜像快照、输入指纹、原始日志、CSV 及完成状态。历史结果校验与新工具执行分别记录；失败尝试保留。已有 Vivado 工程拒绝覆盖，程序镜像变化后建立新工程。

文档源由 architecture.json 组织章节和职责，sections.json 保存正文、表格、公式与稳定锚点，content.json 维护连贯说明。build_core_document.py 生成单一核心 Word，再由 Word 更新目录和引用并检查全页渲染。本文语言整理保持默认 AC1、ISA 及原有流水行为。

```
python -B -X utf8 CPU 核心设计/Vivado/verify.py --suite all
python -B -X utf8 CPU 核心设计/系统/run_system.py --test all
python -B -X utf8 CPU 核心设计/Vivado/run_independent.py
python -B -X utf8 CPU 核心设计/板级/run_matrix.py --candidate ac1
python -B -X utf8 CPU 核心设计/板级/run_matrix.py --candidate forward_onehot
python -B -X utf8 CPU 核心设计/板级/run_matrix.py --candidate wb_mem
python -B -X utf8 CPU 核心设计/文档源/build_core_document.py --independent-run <48 例记录>
python -B -X utf8 CPU 核心设计/文档源/validate_document.py --document <核心 Word>
```

B.2　验证范围与后续修改条件

已有证据覆盖列明的有限输入、程序、逐拍行为、完整板级实现，以及默认 AC1 五程序和停止后 S1 复位。完整算术穷举、全状态形式证明、可变延迟存储、硬件单步、Flash/SD 独立上电启动、直接运行时间和独立频率或功耗测量仍未完成。

源码或镜像变化后，应复跑受影响测试并绑定新指纹。系统仿真 PASS 只表示该入口的有限检查完成；布局布线、时序关闭和实板验收分别以各自报告判断。

性能候选继续隔离，不替换默认 AC1。硬件采集需确认实物板型及完整 JTAG 目标，核对构建身份后再比较两次停止快照；配置下载与 CPU 复位分别控制。

后续修改同步存储延迟或故障响应时，应先重审级间载荷、前递就绪、年龄仲裁与停止状态图。单周期路径仍按实际 setup/hold 约束检查，不能用 false-path 或 multicycle 掩盖功能路径违例。

附录C　参考资料

C.1　课程与工程参考

技术参考采用 AMD UG474 的进位资源说明和 AMD UG903 的时序约束规则。文档组织参考 Ibex Reference Guide 与 CORE-V Core Integration 中的模块、接口和流水描述方法；这些参考不改变本 CPU 的自定义 ISA 及五级实现。

课程方法依据包括 CHA Lec 01-B 的 ISA 到数据通路/CU 流程、CHA Lec07-Sequential circuits analysis and design 的 F/Q/G 及状态转移方法，以及第 7 次课.zip 中的 Lec18 RISC CPU Design-CU design。加减器、HDL、Testbench、性能与 ISA/数据通路内容继续参考课程对应资料及《数字系统设计》第二版第 4 章、第 7 至 8 章。课程串行实验 FSM 用于理解方法，本 CPU 采用本文规定的五级并行、有效位与模式状态机。

第一章为当前 ISA 的唯一指令契约。Pre4 资料用于追溯字段与开发过程，旧 Opcode 和机器字示例的对应关系见第 A.1 节。

板级器件、PL Bank 电平、时钟、按键及 LED 引脚依据 EES331 User Manual v1.0 1220 的 PDF 第 5、7、29 至 30 页；实现工具和约束以 Vivado2025.2 原始报告为准。EES-331 使用 CLG484 封装，本版不采用 PYNQ-Z2 的 CLG400 封装或 125MHz 时钟配置。